PERSONAAI: AN ADAPTIVE AI STUDY ASSISTANT WITH RETRIEVAL-AUGMENTED GENERATION, WEAK-TOPIC MEMORY AND COHORT-BASED COURSE NOTES

A Mini Project Report

submitted in partial fulfilment of the requirements for the award of the degree of

BACHELOR OF TECHNOLOGY

in

ARTIFICIAL INTELLIGENCE

by

Jajula Shivadhanu Goud (23EG106C21)
Kothapally Sai Mani Prasad (23EG106C34)
Baddam Supreeth Reddy (23EG106C05)

Under the guidance of

Dr. M. Trupthi

Professor, Department of Artificial Intelligence



DEPARTMENT OF ARTIFICIAL INTELLIGENCE
ANURAG UNIVERSITY
Venkatapur, Ghatkesar, Hyderabad - 500088

October, 2026

DECLARATION

We hereby declare that the Mini Project report entitled "**PERSONAAI: AN ADAPTIVE AI STUDY ASSISTANT WITH RETRIEVAL-AUGMENTED GENERATION, WEAK-TOPIC MEMORY AND COHORT-BASED COURSE NOTES**" submitted to the Department of Artificial Intelligence, Anurag University, in partial fulfilment of the requirements for the award of the degree of Bachelor of Technology, is a record of original work carried out by us under the guidance of Dr. M. Trupthi, Professor, Department of Artificial Intelligence. The results embodied in this report have not been submitted to any other university or institute for the award of any degree or diploma. Wherever the work of others, including datasets, pretrained models, open source libraries and published papers, has been used, it has been duly acknowledged and cited.

Use of AI tools: We declare that we have used Claude (Anthropic), through the Claude Code assistant, for coding help, debugging, test writing and language editing of this report. Claude Sonnet 5.5 is also a component of the system itself, as described in Chapters 4 and 5. The system design, implementation, testing and interpretation of results are our own, all results reported here were produced by running the submitted code, and we take full responsibility for the content of this report.

Place: Hyderabad
Date: 28-10-2026

Name of the Student	Roll No.	Signature
Jajula Shivadhanu Goud	23EG106C21	
Kothapally Sai Mani Prasad	23EG106C34	
Baddam Supreeth Reddy	23EG106C05	

ANURAG UNIVERSITY | SCHOOL OF ENGINEERING

DEPARTMENT OF ARTIFICIAL INTELLIGENCE

CERTIFICATE

This is to certify that the Mini Project report entitled "**PERSONAAI: AN ADAPTIVE AI STUDY ASSISTANT WITH RETRIEVAL-AUGMENTED GENERATION, WEAK-TOPIC MEMORY AND COHORT-BASED COURSE NOTES**" is a bonafide record of the work carried out by Jajula Shivadhanu Goud (23EG106C21), Kothapally Sai Mani Prasad (23EG106C34) and Baddam Supreeth Reddy (23EG106C05) in partial fulfilment of the requirements for the award of the degree of Bachelor of Technology in Artificial Intelligence during the academic year 2026 to 2027. The work embodied in this report has not been submitted to any other university or institute for the award of any degree or diploma.

Dr. M. Trupthi
Project Guide,
Professor,
Department of Artificial Intelligence

Dr. M. Trupthi,
Head of the Department
Department of Artificial Intelligence

ACKNOWLEDGEMENTS

We express our sincere gratitude to our guide, Dr. M. Trupthi, Professor, Department of Artificial Intelligence, for her valuable guidance, constant encouragement and support throughout this project. Her advice during the reviews on the structure of the report, on presenting every part of the work in clear and detailed paragraphs, and on citing every source properly helped us to describe the system and its evaluation honestly and completely. We are equally grateful to Dr. M. Trupthi, Head of the Department of Artificial Intelligence, for providing the facilities and the academic environment needed to carry out this project, and to the members of the Project Review Committee for their questions and suggestions during the reviews, which helped us to see where the system needed further work.

We are grateful to Prof. V. Vijaya Kumar, Dean, School of Engineering, Dr. Archana Mantri, Vice Chancellor, and the Management of Anurag University for providing the laboratories, computing facilities, library resources and academic environment required to complete this work. We also thank all the faculty and staff of the Department of Artificial Intelligence for the course material and the knowledge on which this project builds, and our families and friends for their patience, encouragement and continued support throughout the semester, without which this project would not have been completed.

Jajula Shivadhanu Goud (23EG106C21)
Kothapally Sai Mani Prasad (23EG106C34)
Baddam Supreeth Reddy (23EG106C05)

ABSTRACT

Students preparing for university examinations revise from scattered lecture notes, slide decks and question banks, much of it scanned. General artificial intelligence chatbots explain concepts fluently, but they answer from general knowledge rather than from the course material, cannot show where an answer came from, and do not remember which topics a student keeps getting wrong. Departments also lack a simple way to give every student of a class the same notes in a form that can be questioned, practised and planned around before an examination timetable.

This project presents PersonaAI, a web-based study assistant built around a class's own course notes. An administrator imports a class list, which creates cohorts, and shares notes by subject and unit; students sign in with their hall ticket number. Word, PowerPoint and document files are split into page-bounded passages, and image-only pages are transcribed by a vision-capable language model. Questions are answered by retrieval-augmented generation with a re-ranker and Claude Sonnet 5.5, and every answer cites its file and page or slide. The same notes produce validated quizzes, a memory of weak topics, adaptive practice and a study plan for a full examination timetable, with calendar export and email reminders.

On 30 held-out Big Data examination questions, notes-grounded answers were fully correct for 23 and partially correct for 5, against 28 correct for the same model without notes; the gap came from passages that retrieval missed and from the system declining to add knowledge the notes lack. On a real cohort of 19 files from five subjects, 371 image-only pages were recovered, all 17 browser checks passed, and every audited quiz contained ten valid questions from the chosen chapter. Grounding answers in a class's notes therefore gives verifiable, syllabus-specific help within seconds.

Keywords: Retrieval-Augmented Generation, Large Language Models, Adaptive Learning, Learner Modelling, Optical Character Recognition, Educational Technology

TABLE OF CONTENTS

List of Figures vi

List of Tables vii

List of Abbreviations viii

CHAPTER 1 INTRODUCTION 1

1.1 Background 1

1.2 Motivation 2

1.3 Problem Statement 2

1.4 Research Objectives 3

1.5 Scope and Limitations 4

1.6 Organization of the Report 5

CHAPTER 2 LITERATURE SURVEY 6

2.1 Review of Relevant Work 6

2.1.1 Retrieval-Augmented Generation 6

2.1.2 Dense Retrieval, Sentence Embeddings and Re-ranking 7

2.1.3 Behaviour and Evaluation of LLM-Based Systems 8

2.1.4 Decomposing Complex Questions 8

2.1.5 Text Recognition in Scanned Course Material 9

2.1.6 Question Generation and LLMs in Education 9

2.1.7 Learner Modelling, Spaced Repetition and Long-Term Memory 10

2.2 Summary of the Literature 10

2.3 Research Gap Identified 12

CHAPTER 3 SYSTEM ANALYSIS AND REQUIREMENTS 13

3.1 Existing Approaches and their Drawbacks 13

3.2 Feasibility Study 14

3.3 Hardware and Compute Used 14

3.4 Software Requirements 15
3.4.1 Functional Requirements 16
3.4.2 Non-Functional Requirements 17
CHAPTER 4 DATASET, DESIGN AND METHODOLOGY 18
4.1 Overall Pipeline 18
4.2 Dataset 19
4.3 Pre-processing 21
4.4 Development / Held-out Split and Avoiding Data Leakage 22
4.5 System Architecture and Model Choice 22
4.6 Methodology 25
4.6.1 Grounded Question Answering with Abstention 26
4.6.2 Multi-Part Questions 27
4.6.3 Quiz Generation and Validation 27
4.6.4 Weak-Topic Memory 27
4.6.5 Adaptive Quiz Allocation 28
4.6.6 Timetable Study Planner, Calendar and Email 28
4.6.7 Cohorts, Accounts and Security 29
4.7 Baseline Models for Comparison 31
4.8 Comparison with Recent (State-of-the-Art) Models 32
CHAPTER 5 IMPLEMENTATION 33
5.1 Training Setup (Inference and Configuration Setup) 33
5.2 Hyper-parameter Tuning Approach 34
5.3 Key Code Snippets 34
5.3.1 Two-Stage Retrieval and Multi-Part Questions 35
5.3.2 Grounding Instructions 36
5.3.3 Reading Image-Only Pages 37
5.3.4 Model Access, Timeouts and Fallback 37

5.3.5 Quiz Validation 38
5.3.6 Learner Model and Planner 39
5.3.7 Account Lockout 40
5.4 Deployment and User Interface 40
5.4.1 Deployment 40
5.4.2 User Interface 40
5.5 Testing 42
5.6 Development Process 44
CHAPTER 6 RESULTS AND DISCUSSION 45
6.1 Evaluation Metrics 45
6.2 Training / Validation Behaviour 46
6.3 Test-Set (Held-Out) Results 47
6.4 End-to-End Verification on the AI Cohort 49
6.5 Comparison with Baselines and Published Work 51
6.5.1 RAG against the LLM without Notes (B1) 51
6.5.2 Adaptive against Even-Mix Quizzes (B4) 52
6.5.3 Relation to Published Work 53
6.6 Error Analysis 54
6.7 Ablation Study 55
6.8 Output Screenshots 56
6.9 Ethical and Responsible AI 62
CHAPTER 7 CONCLUSION AND FUTURE SCOPE 64
7.1 Conclusion 64
7.2 Limitations 65
7.3 Future Enhancements 66
References 67
Appendices 71

Appendix A: GitHub Repository Link and README 71

Appendix B: Trained Model / Weights Link 71

Appendix C: Key Code 72

Appendix D: Plagiarism Report 74

Appendix E: Publications / Certificates 79

LIST OF FIGURES

Fig. No.	Title	Page No.
4.1	Overall pipeline of PersonaAI: data, pre-processing, models, evaluation and deployment	18
4.2	Document ingestion and pre-processing pipeline, including the image-reading stage	21
4.3	Layered system architecture of PersonaAI	23
4.4	Flow of a question through the system, with the multi-part split and the provider chain	24
4.5	Rule-based timetable planner	29
4.6	Sign-in flow with the first-sign-in form and the lockout	30
5.1	Home dashboard of a new student, with the sidebar navigation and profile card	41
6.1	Re-ranker calibration: best passage score for on-syllabus and off-syllabus questions	46
6.2	Correct and partial answers with and without the notes, for both runs and both halves	48
6.3	Pages or slides per file of the AI cohort, and how many were image-only and read by the vision model	51
6.4	Simulated mastery of the genuinely weak topics per quiz	53
6.5	Admin: subjects and the course notes shared with the AI cohort, by unit	57
6.6	First sign-in: one-time sign-up code, email address and a new password	57
6.7	An answer grounded in the notes, with inline citations	58
6.8	A two-part question answered from two subjects' notes	58
6.9	An answer citing PowerPoint slides that names the part the notes do not explain	59
6.10	A question the notes do not cover is declined instead of answered from general knowledge	59
6.11	Quiz results: score, topics to revise and the review of each question with its source	60
6.12	Progress page: score trend and topic accuracy, with topics still being judged after one quiz	60
6.13	Study plan: next examination, statistics, shared-session note and Add to calendar	61
6.14	Home page at phone width	61
D.1	Similarity report: cover page	75
D.2	Similarity report: integrity overview	76
D.3	AI-writing report: cover page	77
D.4	AI-writing report: overview	78

LIST OF TABLES

Table No.	Title	Page No.
1.1	Research objectives and the methodology used to achieve them	4
2.1	Summary of the literature survey	11
3.1	Comparison of existing study tools with PersonaAI	13
3.2	Hardware requirements and development machine	15
3.3	Software used, with versions	15
3.4	Functional requirements	16
3.5	Non-functional requirements	17
4.1	Dataset summary	19
4.2	Files of the AI cohort and what the ingestion pipeline recovered from them	20
4.3	Sample records from the evaluation question set (reference answers shortened)	20
4.4	Model choices and justification	25
4.5	Objective-to-methodology map	25
4.6	Database tables	30
4.7	Baselines used in the evaluation	31
4.8	Components used versus recent alternatives	32
5.1	Inference and configuration setup	33
5.2	How the design parameters were chosen	34
5.3	Pages of the application	41
5.4	Automated test suite (159 tests, all passing)	42
5.5	Sample test cases	43
5.6	Defects found during development and their fixes	43
5.7	Development phases	44
6.1	Answer accuracy with and without the notes in both runs (held-out rows are the test result)	47
6.2	Correct answers by unit (15 questions per unit and set)	48
6.3	Answer length and explicit statements of what the notes do not cover	48
6.4	Browser checks on the AI cohort (headless Chromium, real models)	49
6.5	Chapter quiz audit (10 questions each, Claude Sonnet 5.5)	50
6.6	Observed response and processing times on the development machine	51
6.7	Simulation results: adaptive versus even-mix quizzes (200 simulated students per mode)	53
6.8	Notes-grounded answers of the Claude run that were not fully correct	54
6.9	Ablation study	55
7.1	Objective vs Methodology vs Implementation vs Result	64
B.1	Pretrained models used	72
C.1	Modules and their responsibilities	73

LIST OF ABBREVIATIONS

Abbreviation	Expansion
AI	Artificial Intelligence
API	Application Programming Interface
AUC	Area Under the (ROC) Curve
BKT	Bayesian Knowledge Tracing
BM25	Best Matching 25 (keyword ranking function)
CPU	Central Processing Unit
CSV	Comma-Separated Values
DKT	Deep Knowledge Tracing
DOCX	Office Open XML word-processing document
DPR	Dense Passage Retrieval
FR	Functional Requirement
GPU	Graphics Processing Unit
HLR	Half-Life Regression
ICS	iCalendar file format
IEEE	Institute of Electrical and Electronics Engineers
JSON	JavaScript Object Notation
LLM	Large Language Model
MCQ	Multiple-Choice Question
MRR	Mean Reciprocal Rank
NFR	Non-Functional Requirement
NLP	Natural Language Processing
OCR	Optical Character Recognition
PBKDF2	Password-Based Key Derivation Function 2
PDF	Portable Document Format
PPTX	Office Open XML presentation (PowerPoint)
Q&A	Question and Answer
RAG	Retrieval-Augmented Generation
RAM	Random Access Memory
SDG	Sustainable Development Goal
SMTP	Simple Mail Transfer Protocol
SOTA	State of the Art
SQL	Structured Query Language
UI	User Interface

CHAPTER 1
INTRODUCTION

1.1 Background

Large language models (LLMs) have made conversational artificial intelligence freely available to students. They explain concepts fluently, answer follow-up questions at any hour and can produce worked examples on demand, which makes them attractive as study aids [1]. However, an LLM answers from its general training data, and that data may not match how a particular course defines, scopes or teaches a topic. The model also occasionally produces plausible but incorrect statements, a behaviour known as hallucination [2], and a student revising the night before an examination has no easy way to tell a correct sentence from an invented one. Finally, a general assistant keeps no structured record of an individual learner's performance, so a student who repeatedly struggles with one topic receives the same generic help as everyone else, and the assistant cannot tell the student which parts of the syllabus still need attention.

Retrieval-Augmented Generation (RAG) addresses the first two problems by retrieving relevant passages from a document collection and asking the model to answer from them, so that each answer can be traced back to its source [3]. Research on learning addresses the third problem: practice testing and spaced review are among the most effective study techniques known [4], [5], [6], and they are most useful when practice is directed at what the learner does not yet know. A study assistant that combines both ideas can therefore answer from the course material, show where every answer came from, and use the student's own quiz results to decide what to practise next and how to divide the remaining days before an examination. These ideas are well established individually, but they are rarely combined in one tool that a whole class can use.

PersonaAI brings these ideas together in a single web application designed for a university department. An administrator imports the class list and shares each subject's notes, unit by unit, with the students of a cohort, who sign in with their hall ticket numbers. The application grounds every answer in those notes and cites the file and page or slide; it turns the same notes into multiple-choice quizzes; it remembers which topics each student answers incorrectly; and it uses that memory to focus later quizzes and to build a day-by-day revision plan for the student's complete examination

timetable, which can be added to a phone calendar and emailed each morning. The system was evaluated on examination questions from a Big Data course and demonstrated on a third-year Artificial Intelligence cohort whose notes cover five subjects in nineteen PDF, Word and PowerPoint files.

1.2 Motivation

Examination revision usually happens from several sources at once: unit-wise lecture notes, slide decks shared by different faculty, viva question lists and previous question papers. Finding the page that answers a specific question takes time, and much of the material in an engineering department is distributed as scanned PDFs or picture-heavy slides that cannot even be searched. Answers that do not match the syllabus can also mislead students, because an examination expects the points that were taught in the course rather than a generic textbook answer. While inspecting the AI cohort's notes for this project, we found that 371 pages and slides contained no extractable text at all, so a keyword search or an ordinary RAG system would have treated most of three NLP units as empty. This observation motivated the image-reading stage described in Chapter 4.

A second motivation is that practice testing and spaced review are effective [6], yet organising them by hand across dozens of topics and several subjects is tedious, so students rarely do it, and they often do not know which topics they are weak in until they lose marks. Examination timetables in our university place five or six papers within two weeks, which makes planning even harder. Many students also cannot rely on paid AI services or a continuous internet connection, so a tool that falls back to a local model and works on a phone browser is more inclusive. This connects the project to the United Nations Sustainable Development Goal 4, Quality Education, which calls for inclusive and equitable learning opportunities for all [7]. Finally, a department benefits from a single place where every student of a class receives the same notes in a form they can question.

1.3 Problem Statement

The problem addressed by this project is to design, build and evaluate a web application in which a department shares course notes with a cohort of students, and each student can ask questions that are answered only from those notes, with verifiable file-and-page or slide citations and an honest statement when the notes do not contain the answer.

The same material must be turned into validated multiple-choice quizzes; the application must maintain a per-student, per-topic model of strengths and weaknesses from quiz results; and it must use that model to target later quizzes and to produce a day-by-day study plan for an examination timetable with several subjects. The solution has to read PDF, Word and PowerPoint files including image-only pages, keep each cohort's notes and each student's data separate, protect accounts with passwords and one-time codes, and remain usable when the online AI service is slow, rate-limited or unavailable.

1.4 Research Objectives

The research objectives of the project are stated below as O1 to O9. Each objective is mapped to its methodology in Table 1.1, developed in Chapter 4, implemented in Chapter 5 and assessed against its measured result in Table 7.1.

To construct an ingestion pipeline that reads PDF, Word and PowerPoint notes page by page, recovers image-only pages with a vision-capable model, and stores page-bounded passages in a vector index tagged with the cohort, file and page (O1).

To develop a question-answering module that answers only from the retrieved notes, cites the file and page of every statement, answers multi-part questions part by part, and says so when the notes do not contain the answer (O2).

To generate valid multiple-choice quizzes of five, ten or fifteen questions from a chosen subject, chapter or set of topics, with an answer key, explanation and source for every question, saving every answer with its topic (O3).

To track each student's accuracy on each topic over the most recent answers, classify every topic as weak, learning, strong or not yet tested, and show this with the trend of quiz scores on the progress page (O4).

To provide an adaptive quiz mode that places most questions on weak topics, then on untested topics and strong topics not seen for the longest time, and to compare it with an even spread of questions (O5).

To develop a rule-based study planner that divides the days before each examination between subjects by weakness and urgency, reserves the day before every examination for revision, and exports the plan to a calendar and a morning email (O6).

To construct a cohort-based account system in which an administrator imports class lists, shares notes per subject and unit and issues one-time sign-up codes, with hashed passwords, lockout after failed sign-ins and strict cohort isolation (O7).

To evaluate the system honestly by comparing answers with and without the notes on held-out examination questions using a blind judge, and by verifying the complete application end to end on a real cohort in a browser (O8).

To provide a clean, responsive interface that works on desktop and phone browsers and keeps working when the online model fails, by falling back from Claude to Gemini and then to a local model, without exposing any keys or passwords (O9).

Table 1.1 Research objectives and the methodology used to achieve them

Obj.	Objective (short)	Methodology
O1	Ingest notes, including image-only pages	Page/slide extraction; Claude vision for pages under 25 words; header/footer removal; 180/40-word page-bounded chunks; MiniLM embeddings in ChromaDB; topic extraction with a coverage filter
O2	Grounded, cited, multi-part answers	LLM question splitting; per-part retrieval of 20 candidates, cross-encoder re-ranking (threshold −5), interleaved merge of up to 8 excerpts; grounded prompt with [S#] tags and partial-answer rules
O3	Valid quizzes by chapter	Chapter picker; topic allocation; one structured-output call (JSON schema with enumerated topic tags); rule validation; regeneration; option shuffling
O4	Weak-topic memory	Accuracy over the last 10 answers per topic; weak < 60 %, strong ≥ 80 %, judged after 3 answers; scores rebuilt by one SQL statement after every quiz
O5	Adaptive quizzes	About 60 % of questions on weak topics, rest to untested, low-scoring and least-recent strong topics; 200-student simulation against an even mix
O6	Timetable study plan, calendar and email	Weighted largest-remainder allocation of 30-minute slots; day shares by weight ÷ days left; revision days; shared sessions when time is short; iCalendar export; SMTP email
O7	Cohorts and secure accounts	Class-list import; cohorts; shared notes per subject/unit; PBKDF2 hashes; one-time codes; 5 failures → 5-minute lockout; cohort filters in SQL and the vector index
O8	Honest evaluation	Development/held-out split, no-notes baseline, blind LLM judge (Gemini and Claude runs); Playwright browser check of the AI cohort; chapter quiz audit
O9	Resilient, professional interface	Streamlit multipage app, custom theme with line icons; provider chain Claude → Gemini → Flash-Lite → Ollama with timeouts, one retry and logged failures

1.5 Scope and Limitations

The scope of the project covers the course notes of one or more cohorts, each with several subjects and units; private notes that an individual student adds; a private, per-

student memory of quiz results; the five student pages (Home, Notes Q&A, Quiz, Progress and Study Plan) and an administration page; email updates and calendar export; an offline mode that uses a local model; and a quantitative evaluation of the approach. Lecture recording and transcription, a native mobile application, payments, timetable import from the university portal and deployment across several institutions are outside the scope. The application is intended for a single department running it on its own computer or a small server, and the design choices described in Chapter 4 favour simple, explainable components that run on a laptop over large models that would need dedicated hardware.

The main limitations are discussed in Chapters 6 and 7. No model was trained or fine-tuned; every model is pretrained and used for inference only, so the project's contribution lies in the pipeline, the rules and the evaluation rather than in new model weights. Retrieval still misses some answers when a passage loses its section heading, and in one observed case a definition on a slide ranked just below the relevance threshold. The accuracy evaluation uses one course and sixty questions, and the benefit of adaptive quizzes was measured by simulation rather than with real students over a semester. Pictures that contain no text, such as decorative images on slides, cannot be turned into searchable content, and the hall-ticket format of the demonstration cohort was invented for the demonstration rather than taken from university records.

1.6 Organization of the Report

Chapter 2 reviews the literature on retrieval-augmented generation, retrieval and re-ranking, the evaluation of LLM-based systems, question decomposition, document text recognition, question generation and learner modelling, and it identifies the research gap that the project addresses. Chapter 3 analyses existing approaches and the feasibility of the project and lists the hardware, software and requirements. Chapter 4 describes the two datasets, the overall pipeline, the architecture, the methodology for each objective, the baselines and the recent alternative models. Chapter 5 covers the implementation: configuration, parameter selection, key code, the user interface, testing and the development process. Chapter 6 presents the evaluation results, the end-to-end verification on a real cohort, the error analysis, the ablation experiments, screenshots of the running application and a discussion of ethics, and Chapter 7 concludes the report against each objective and outlines the future work.

CHAPTER 2
LITERATURE SURVEY

This chapter reviews the work that PersonaAI builds on, in seven areas: retrieval-augmented generation; dense retrieval, sentence embeddings and re-ranking; the behaviour and evaluation of LLM-based systems; the decomposition of complex questions; text recognition in scanned documents; automatic question generation and LLMs in education; and learner modelling, spaced repetition and long-term memory. For each area the most relevant papers are summarised together with their datasets, results and limitations, and the way in which PersonaAI uses or departs from them is explained. A summary of the reviewed work is given in Table 2.1 in the format required by the department, and the research gap that follows from the review is stated in Section 2.3. The review concentrates on peer-reviewed conference and journal papers, supplemented by the official documentation of the software components that the system depends on.

2.1 Review of Relevant Work

2.1.1 Retrieval-Augmented Generation

Lewis et al. [3] introduced Retrieval-Augmented Generation, in which a neural retriever selects passages from a large document collection and a sequence-to-sequence generator conditions its answer on them. Trained end to end on Wikipedia, RAG set the state of the art on three open-domain question-answering tasks at the time and produced more specific and factual text than a model that relied only on its parameters. RAG also makes answers traceable to their sources and can be updated by changing the collection rather than retraining the model, which is exactly the property a course assistant needs when the notes change from one semester to the next. PersonaAI applies the same idea with a cohort's course notes as the collection, but it uses an instruction-following LLM without any fine-tuning and adds explicit instructions to cite every statement and to refuse, fully or partly, when the retrieved passages do not contain the answer.

Gao et al. [8] survey the rapidly growing RAG literature and organise it into naive RAG, in which the system retrieves and then generates; advanced RAG, which adds improved indexing, query rewriting and re-ranking; and modular RAG, which builds pipelines from interchangeable components. In this taxonomy PersonaAI is an advanced RAG

system with a re-ranking stage and a query-splitting stage. Asai et al. [9] proposed Self-RAG, which fine-tunes a model to decide when to retrieve and to critique its own output with special reflection tokens; their 7B and 13B models outperformed ChatGPT and retrieval-augmented Llama 2-chat on several question-answering and fact-verification tasks and improved citation accuracy. Self-RAG requires fine-tuning the generator, which is not possible with a hosted API model, so PersonaAI implements a simpler form of self-assessment: a score threshold on the re-ranker before generation and a NOT_FOUND reply from the generator after it.

2.1.2 Dense Retrieval, Sentence Embeddings and Re-ranking

Karpukhin et al. [10] showed that a dual encoder trained on question–passage pairs, known as Dense Passage Retrieval, outperforms the keyword-based BM25 ranking function [11] by 9 to 19 percentage points in top-20 passage-retrieval accuracy on open-domain benchmarks. Sentence-BERT [12] showed that Siamese transformer networks produce sentence embeddings whose cosine similarity reflects meaning, which enables fast semantic search, and MiniLM [13] distils large transformers into small, fast models. The all-MiniLM-L6-v2 model used in PersonaAI combines these results: it produces 384-dimensional embeddings and runs comfortably on a laptop processor. One limitation, observed during this project, is that it reads at most 256 word-pieces, so longer passages are silently truncated, which is why the passages in PersonaAI are limited to 180 words. A second limitation is that a small embedding model can rank a definition low when it does not repeat the words of the question, which motivates the re-ranking stage.

Embedding search encodes the question and each passage independently. A cross-encoder instead reads the question and the passage together and is far more accurate at judging relevance, at a higher cost for each pair. Nogueira and Cho [14] re-ranked BM25 candidates with a BERT cross-encoder trained on the MS MARCO passage-ranking dataset [15] and improved the previous state of the art by 27 per cent, relative, in MRR@10. PersonaAI uses this two-stage design: embedding search returns twenty candidates for each question or question part, and a MiniLM cross-encoder trained on MS MARCO selects the best five. Recent general-purpose embedding models such as E5 [16] and BGE [17] are larger and stronger alternatives for the first stage, and they are compared with the chosen components in Section 4.8, together with the reasons why the smaller models were kept for a system that has to run on an ordinary laptop.

2.1.3 Behaviour and Evaluation of LLM-Based Systems

Ji et al. [2] survey hallucination in natural language generation, distinguishing intrinsic hallucinations, which contradict the source, from extrinsic ones, which add content that cannot be verified, and they list retrieval grounding among the main mitigation strategies. Liu et al. [18] showed that LLMs use long contexts unevenly: accuracy on multi-document question answering is highest when the relevant passage is at the beginning or the end of the context and drops when it is in the middle. This supports giving the model a small number of well-ranked passages, five for a single question and at most eight for a multi-part question in PersonaAI, rather than many weakly related ones. It also explains why the merge step for multi-part questions interleaves the best passages of each part instead of appending all passages for the first part before those for the second, which would push the second part's evidence into the middle of the context.

Evaluating free-text answers is expensive when it is done by hand. Zheng et al. [19] studied strong LLMs as judges on MT-Bench and Chatbot Arena and found that GPT-4's judgements agreed with human preferences more than 80 per cent of the time, similar to the agreement between humans, while also documenting position, verbosity and self-enhancement biases. Es et al. [20] proposed RAGAs, a set of reference-free metrics for faithfulness, answer relevance and context relevance that are computed with an LLM and were validated against human judgements on their WikiEval dataset. PersonaAI's evaluation follows these lessons: the judge grades each answer against a written reference answer from the course question bank, the two candidate answers are shown in random order without labels, and columns are provided in the results files for manual overrides of any verdict.

2.1.4 Decomposing Complex Questions

Students often ask two things at once, for example how two concepts from different units differ or what two components each do. Press et al. [21] measured the compositionality gap of language models, the fraction of two-part questions that a model answers wrongly even though it answers both parts correctly when they are asked separately, and showed that the gap does not shrink as models grow. Their self-ask method narrows it by having the model state and answer explicit follow-up questions before giving the final answer. PersonaAI meets the same problem on the retrieval side rather than the reasoning side: when one search is run for a two-part question, the

passages for one part can take every slot and the other part is reported as missing. The system therefore asks the LLM to split such questions into standalone parts, searches for each part separately and merges the results, which fixed a cross-subject question in our verification that had previously cited only one subject.

2.1.5 Text Recognition in Scanned Course Material

Optical character recognition has a long history; the Tesseract engine described by Smith [22] finds text lines, recognises words with an adaptive classifier and became one of the most widely used open-source recognisers. Classical engines work well on clean printed pages but struggle with the mix of photographs of whiteboards, screenshots, diagrams with labels and mathematical notation found in lecture material, and they return text without any understanding of the layout. Multimodal language models such as Gemini [23] and Claude [24] accept page images directly and can be asked to transcribe only the readable text and to ignore decoration, returning the result in a structured format. PersonaAI uses this capability only for pages and slides that contain fewer than twenty-five extractable words, sending them in batches of six with a JSON schema that maps each original page number to its text, so that citations still point at the right page of the original file.

2.1.6 Question Generation and LLMs in Education

Kurdi et al. [25] systematically reviewed automatic question generation for educational purposes, covering studies published between 2015 and 2019. They found that most systems generate factual questions from text, that evaluation practice is inconsistent across studies, and that controlling question difficulty remains an open problem. These systems pre-date instruction-tuned LLMs, which can now write complete multiple-choice questions with distractors and explanations, but whose answer keys and distractors still need checking. Kasneci et al. [1] discuss the opportunities of LLMs in education, including personalised explanations, feedback and practice material, together with risks such as incorrect output, bias and over-reliance, and argue for grounding and human oversight. PersonaAI's quiz module responds to both points: questions are generated only from the chosen chapter's notes, every question carries its source, the output must match a strict schema, and rule-based validation rejects questions with duplicate options or unknown topics.

2.1.7 Learner Modelling, Spaced Repetition and Long-Term Memory

Intelligent tutoring systems estimate what a learner knows in order to choose the next activity. Bayesian Knowledge Tracing [26] models the probability that a student has mastered each skill as a hidden state that is updated after every answer. Deep Knowledge Tracing [27] replaced this with a recurrent neural network and reported an area under the curve of 0.86 against 0.67 for Bayesian Knowledge Tracing on the ASSISTments dataset, at the cost of needing large interaction logs and being harder to interpret. Settles and Meeder [28] introduced half-life regression, which estimates how long a word stays in a learner's memory; on Duolingo practice logs it reduced the error in predicting recall by more than 45 per cent compared with baseline methods. Each of these models needs many answers per student before it becomes reliable, which a new cohort at the start of a semester does not yet have, so a simpler and explainable model is used in PersonaAI.

Roediger and Karpicke [4] showed that taking tests improves long-term retention more than restudying the same material, an effect known as the testing effect, and Cepeda et al. [5] found across hundreds of experiments that spacing practice over time improves recall compared with massed practice. Dunlosky et al. [6] rated practice testing and distributed practice as the two most useful of ten common study techniques. Finally, recent work on long-term memory for chat assistants, such as the LongMemEval benchmark [29], shows that commercial assistants and long-context LLMs lose about 30 per cent accuracy when information must be recalled across sustained interactions. PersonaAI does not try to remember conversations; its long-term memory is a structured, per-topic record of quiz performance stored in a database, which is simple, explainable to the student and incapable of hallucinating a result that never happened.

2.2 Summary of the Literature

Table 2.1 summarises the reviewed papers in the format required by the department, giving for each the author and year, the journal or venue, the method, the dataset, the main result and the main limitation. The table shows a clear pattern. The retrieval and generation papers report results on large public benchmarks built from Wikipedia or web search logs, the evaluation papers propose general methods for judging answers, and the learner-modelling papers rely on large logs collected over years of use. None of them evaluates a complete study assistant on the notes and examination questions of a single university course, and none connects the answers a student receives with the

questions they get wrong and the way their revision time is planned. These observations lead directly to the research gap stated in Section 2.3.

Table 2.1 Summary of the literature survey

Author / Year	Journal / Venue	Method	Dataset	Results	Limitations
Lewis et al., 2020 [3]	NeurIPS	Neural retriever + seq2seq generator trained end to end	NQ, TriviaQA, WebQuestions, CuratedTrec, MS MARCO, FEVER	State of the art on three open-domain QA tasks; more factual generation	Wikipedia-scale index; joint training; no explicit abstention
Karpukhin et al., 2020 [10]	EMNLP	Dual-encoder dense passage retrieval	NQ, TriviaQA, WebQ, CuratedTREC, SQuAD	+9–19 points top-20 retrieval accuracy over BM25	Needs labelled question–passage pairs
Nogueira & Cho, 2019 [14]	arXiv	BERT cross-encoder re-ranking of BM25 candidates	MS MARCO, TREC-CAR	27 % relative gain in MRR@10 on MS MARCO	Cost grows with candidates
Gao et al., 2023 [8]	arXiv (survey)	Survey of naive, advanced and modular RAG	Literature	Taxonomy of retrieval, augmentation and generation methods	No new experiments
Asai et al., 2024 [9]	ICLR	Self-RAG: retrieve on demand and critique output	Open-domain QA, reasoning, fact verification	Beat ChatGPT and retrieval-augmented Llama 2-chat on several tasks	Requires fine-tuning the generator
Liu et al., 2024 [18]	TACL	Study of where relevant information sits in the context	Multi-document QA, key-value retrieval	Accuracy highest at start/end, lower in the middle	Analysis only
Press et al., 2023 [21]	Findings of EMNLP	Compositionality gap; self-ask decomposition	2WikiMultiHop, Musique, Compositional Celebrities	Gap does not shrink with scale; self-ask narrows it	Reasoning only; no retrieval of course notes
Smith, 2007 [22]	ICDAR	Tesseract OCR: line finding, adaptive word recognition	Printed document images	Widely used open-source OCR engine	Weak on photos, diagrams, mixed layouts
Es et al., 2024 [20]	EACL (Demos)	Reference-free LLM metrics for RAG	WikiEval	Faithfulness and relevance agree with human judgements	Depends on the scoring LLM
Zheng et al., 2023 [19]	NeurIPS (D&B)	Strong LLMs as judges	MT-Bench, Chatbot Arena	Over 80 % agreement with humans	Position, verbosity, self-enhancement bias
Ji et al., 2023 [2]	ACM Computing Surveys	Survey of hallucination in NLG	Literature	Taxonomy, metrics and mitigation	Largely pre-dates current LLMs
Kasneci et al., 2023 [1]	Learning and Individual Differences	Position paper on LLMs in education	—	Opportunities and risks; calls for oversight	No empirical evaluation
Kurdi et al., 2020 [25]	Int. J. AI in Education	Systematic review of question generation	Studies 2015–2019	Mostly factual questions; inconsistent evaluation	Pre-LLM systems

Author / Year	Journal / Venue	Method	Dataset	Results	Limitations
Piech et al., 2015 [27]	NeurIPS	Recurrent network knowledge tracing (DKT)	ASSISTments, Khan Academy	AUC 0.86 vs 0.67 for BKT	Needs large logs; opaque
Settles & Meeder, 2016 [28]	ACL	Half-life regression for spaced repetition	Duolingo practice logs	Over 45 % lower recall-prediction error	Word-level, language-learning domain
Wu et al., 2025 [29]	ICLR	Benchmark of long-term chat memory	500 questions over long chat histories	About 30 % accuracy drop over sustained interactions	Conversational memory, not learner performance

2.3 Research Gap Identified

The first gap concerns the combination of features. Retrieval-augmented systems answer questions, question-generation systems write questions, and learner models and spaced-repetition schedulers decide what to practise, but few student-facing tools connect all three so that the questions a student gets wrong change what they are quizzed on next and how their revision time is planned across a full examination timetable. A related gap is organisational: the reviewed systems assume an individual user who collects their own documents, whereas in a university the notes are produced by faculty and shared with a whole class. A tool that a department can run for a cohort, with shared notes per subject and unit, private progress for every student and proper accounts, is missing from the reviewed literature, and the project addresses this combination at the scale of a mini project.

The second gap concerns evidence and resources. Published RAG results are mostly on Wikipedia-based benchmarks, and there is little evidence on how a small, laptop-friendly pipeline performs on a single course's own notes and examination questions, compared with the same LLM answering without notes, with abstentions counted rather than hidden. Real departmental notes are also far messier than benchmark text: a large share of the pages are scans or picture-only slides, which an ordinary RAG pipeline silently treats as empty. At the same time, methods such as Self-RAG or Deep Knowledge Tracing need fine-tuning or large datasets, while a study assistant needs components that run on a laptop or a modest API budget with an offline fallback, and a learner model whose decisions can be explained to the student in one sentence. PersonaAI is designed and evaluated around exactly these constraints.

CHAPTER 3
SYSTEM ANALYSIS AND REQUIREMENTS

3.1 Existing Approaches and their Drawbacks

Without a dedicated tool, a student revises by reading PDFs and slide decks, searching them for keywords, asking a general chatbot and occasionally attempting question-bank questions, while the department shares notes through messaging groups or a file drive. The drawbacks are that scanned pages cannot be searched, answers from a chatbot are not tied to the syllabus, there is no record of which topics were answered wrongly, practice is not targeted, and revision across five or six papers in two weeks is planned by hand. Table 3.1 compares the main categories of existing tools with the proposed system on the features that matter for this problem. The comparison reflects the general design of each category at the time of writing; individual products change frequently, and some offer partial versions of the listed features, so the table should be read as a summary of typical behaviour rather than as a review of any single product.

Table 3.1 Comparison of existing study tools with PersonaAI

Tool / approach	Answers from class notes	Citations	Quizzes	Tracks weak topics	Study plan	Class-wide sharing
General AI chatbots	No (general knowledge)	No	On request	No	Generic	No
Notes-grounded assistants	Own uploads	Yes	Limited	No	No	Limited
Flashcard apps	Cards written by the user	—	Flashcards	Per card	Review schedule	Shared decks
Quiz platforms	From the user's sets	—	Yes	Limited	No	Classes
Learning management systems	Files only	—	Manual	Gradebook	No	Yes
PersonaAI (this project)	**Yes, incl. scanned pages**	**File + page/slide**	**Auto MCQs by chapter**	**Per topic**	**Exam timetable, calendar, email**	**Cohorts by subject and unit**

The gap that PersonaAI addresses is therefore the combination, in one tool, of notes-grounded answers that also work on scanned material, automatic quizzes by chapter, and a learner model that feeds back into both quizzes and planning, offered to a whole cohort with proper accounts and remaining usable when the online model fails. Learning management systems already handle class-wide sharing, but they treat notes

as files to download rather than as knowledge to question. General chatbots handle explanation well but cannot be restricted to a course, and quiz platforms and flashcard applications require students or teachers to write every question by hand. The proposed system takes the sharing model of a learning management system, the explanation ability of a chatbot restricted to the notes, and the practice loop of a quiz platform, and adds a planner that ties them to the examination dates that actually matter to the student.

3.2 Feasibility Study

Technical feasibility was established by building the system entirely from mature open-source libraries and documented APIs: Streamlit for the web interface [30], ChromaDB for vector search [31], sentence-transformers for local embeddings and re-ranking [32], SQLite for application data, the Claude API for generation, structured output and image reading [24], the Gemini API as an online fallback [33], and Ollama for local models [34]. The embedding and re-ranking models run on a laptop processor, and the local Llama model runs on a laptop graphics card. Every component was exercised by automated tests and by an end-to-end browser check, so there is no part of the design whose feasibility rests on an assumption. Reading PowerPoint files required the python-pptx library, and old .ppt files are converted with LibreOffice when it is installed.

Economic feasibility was assessed from the measured token usage of the system. Reading the 371 image-only pages of the demonstration cohort used about 602,000 input tokens and 93,000 output tokens, which cost about 2.10 US dollars once for the whole cohort. An answer uses a few thousand input tokens and a ten-question quiz about ten thousand, so a class of sixty students asking twenty questions and taking five quizzes each per week costs a few US dollars per week. When no paid key is available, the system falls back to the free tier of the Gemini API and then to a local model, so the project requires no paid service to run. Operationally, students need only a web browser on a laptop or phone, while the administrator imports a CSV file and uploads notes. In terms of schedule, the work was planned in phases, each ending with a working, tested and committed version (Section 5.6).

3.3 Hardware and Compute Used

No model was trained or fine-tuned in this project, so no training time on a graphics processor was needed. All compute was used for inference: embedding and re-ranking on the laptop processor, local LLM generation on the laptop graphics card through

Ollama when the online models are unavailable, and online generation and image reading through the Claude and Gemini APIs. Indexing the nineteen files of the demonstration cohort, including the image-reading stage, took about 335 seconds in total, and loading the embedding and re-ranking models takes about thirty seconds once after the server starts. Table 3.2 lists the minimum requirements for running the application and the development machine that was actually used; the observed run times of the main operations are reported in Chapter 6, where they are measured on the same machine through a real browser.

Table 3.2 Hardware requirements and development machine

Item	Minimum	Development machine used
Processor	Intel Core i5 or equivalent	AMD Ryzen 7 7445HS
RAM	8 GB (16 GB for offline mode)	16 GB
GPU	Not required (speeds up offline mode)	NVIDIA GeForce RTX 4050 Laptop GPU (used by Ollama)
Operating system	Windows 10/11, Linux or macOS	Windows 11 Home
Network	Needed for the Claude and Gemini APIs; not needed offline	Broadband
Training time	Not applicable (pretrained models only)	Not applicable

3.4 Software Requirements

Table 3.3 lists the software used, with the versions installed in the project's virtual environment, together with the development environment. All versions were read from the installed packages rather than from memory, so that the results can be reproduced with the same environment. The development environment consisted of Visual Studio Code with the Claude Code assistant, a Python virtual environment, Git for version control with the repository published on GitHub, and Playwright with a headless Chromium browser for end-to-end checks. The functional requirements derived from the problem statement are listed in Table 3.4 with the objective that each one serves, and the non-functional requirements are listed in Table 3.5, which together define what the implementation in Chapter 5 must achieve and what the evaluation in Chapter 6 must verify.

Table 3.3 Software used, with versions

Component	Technology	Version
Language	Python	3.12.10

Component	Technology	Version
Web UI	Streamlit (custom theme, st.navigation)	1.65.0
LLM (primary)	Claude Sonnet 5.5 (claude-sonnet-5-5) via the anthropic SDK	2.3.0
LLM (fallback)	Gemini API via google-genai (gemini-3.8-flash, gemini-3.5-flash-lite)	2.28.0
LLM (offline)	Ollama with llama3.1:8b; ollama Python client	0.6.3
Embeddings and re-ranking	sentence-transformers: all-MiniLM-L6-v2, cross-encoder/ms-marco-MiniLM-L-6-v2	6.1.0
Deep learning runtime	PyTorch	2.14.1
Vector database	ChromaDB (persistent, cosine distance)	1.5.9
Application database	SQLite (Python standard library)	—
Document parsing	pypdf; python-docx; python-pptx; Pillow	6.19.0; 1.2.0; 1.0.2; 12.3.0
Charts and data	Plotly; pandas	7.1.0; 3.0.6
Testing	pytest with Streamlit AppTest; Playwright (headless Chromium)	9.1.1; 1.64.1
Development environment	Visual Studio Code, Claude Code, Python venv, Windows 11	—
Version control	Git, GitHub	2.52.0

3.4.1 Functional Requirements

Table 3.4 Functional requirements

ID	Requirement	Obj.
FR1	The admin imports a class list (hall ticket, name, department, year); each department and year becomes a cohort.	O7
FR2	The admin adds subjects to a cohort and shares notes per subject and unit; all students of the cohort see them.	O7
FR3	The admin issues one-time sign-up codes and resets passwords; codes are shown once and stored only as hashes.	O7
FR4	A student signs in with hall ticket and password; the first time with the code, an email address and a new password.	O7
FR5	Five failed sign-ins within five minutes lock the account for five minutes.	O7
FR6	Uploads accept PDF, DOCX, PPTX, PPT and TXT; image-only pages and slides are read by a vision model.	O1
FR7	The system extracts the topics the notes explain; students may also upload private notes.	O1
FR8	Answers come only from the notes, cite file and page/slide, handle multi-part questions and say what is missing.	O2
FR9	Quizzes of 5, 10 or 15 MCQs are generated by subject, chapter or topic, in adaptive or even-mix mode, and graded.	O3, O5
FR10	Every answer is saved with its topic; topic scores and weak/strong status are updated after each quiz.	O4
FR11	A student creates a plan from an exam timetable and hours per day, marks days done, regenerates it and exports it.	O6
FR12	Students choose email updates: quiz results and a morning study-plan email.	O6
FR13	If a model fails, the next is tried: Claude → Gemini → Gemini Flash-Lite → Ollama, each with a timeout.	O9

3.4.2 Non-Functional Requirements

Table 3.5 Non-functional requirements

Category	Requirement
Accuracy and honesty	Answers must be traceable to the notes; the assistant must say what the notes do not contain.
Performance	An answer within about 10 s; a 10-question quiz within about 25 s; upload processed in the background of one page view.
Availability	Usable when the online API is slow, rate-limited or offline (fallback chain, local model).
Security	Passwords as salted PBKDF2 hashes; one-time codes; lockout; keys only in a git-ignored .env; never logged.
Privacy and isolation	Every query filtered by user and cohort; one cohort never sees another cohort's notes.
Usability	Professional, consistent interface; works at desktop and phone widths; clear empty states and errors.
Maintainability	Interface separate from logic; one module per responsibility; 159 automated tests.
Portability	Runs with "streamlit run app.py" on Windows, Linux or macOS with Python 3.12.

CHAPTER 4
DATASET, DESIGN AND METHODOLOGY

4.1 Overall Pipeline

Fig. 4.1 shows the complete pipeline in three lanes. Lane A runs once for every uploaded file: text is extracted page by page or slide by slide, pages that are only images are read by a vision model, the text is cleaned, split into page-bounded passages called chunks, embedded and stored in ChromaDB with the owner, cohort, file and page of each chunk. Lane B runs for every question, quiz or plan: questions with several parts are split, the most relevant chunks are retrieved and re-ranked, and an LLM produces a cited answer or a set of validated quiz questions, whose results update the student's topic scores, which in turn drive adaptive quizzes, the study plan and the email updates. Lane C is the evaluation and deployment: questions from the course question bank, which is deliberately kept out of the index, are answered with and without the notes and graded by a blind judge, the whole application is checked in a headless browser, and the code is published on GitHub.

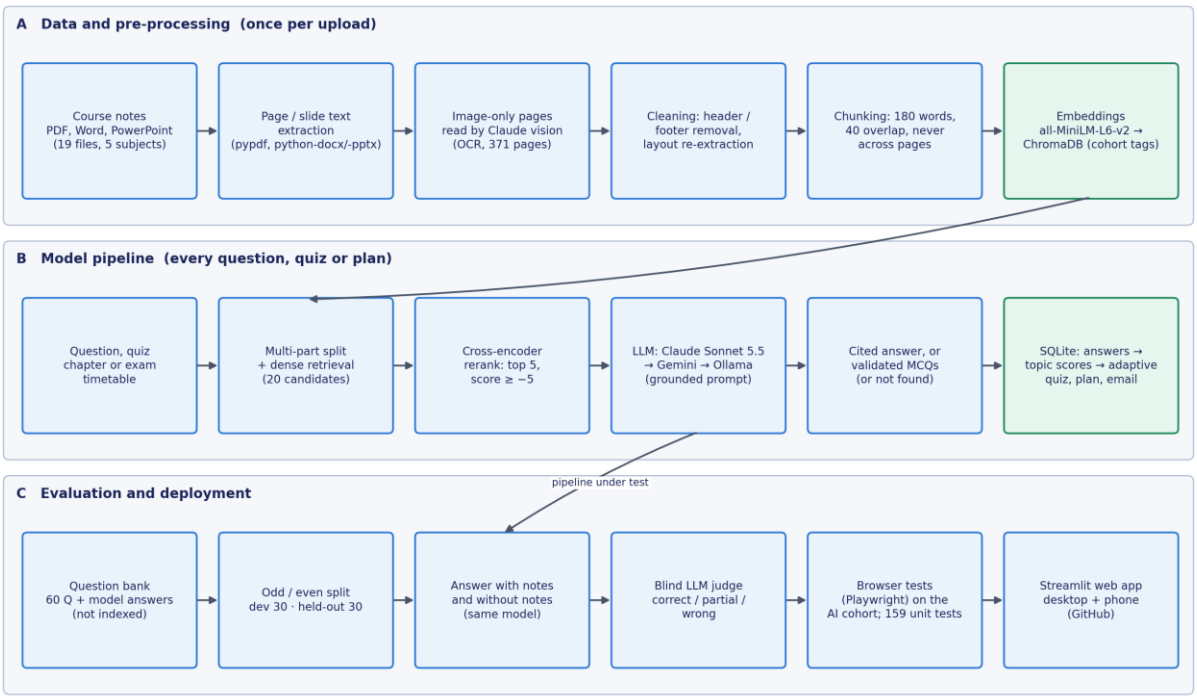


Fig. 4.1 Overall pipeline of PersonaAI: data, pre-processing, models, evaluation and deployment

4.2 Dataset

PersonaAI does not train a model, so its data has two roles rather than training, validation and test splits. The first dataset is the course material that the system retrieves from. For the demonstration and the end-to-end verification this is the real notes of a third-year Artificial Intelligence cohort: nineteen files covering five subjects, namely Natural Language Processing, Deep Learning, Big Data, Cyber Security and Negotiation Skills, in PDF, Word and PowerPoint formats. The second dataset is the evaluation question set used to measure answer quality, which consists of sixty short examination questions with reference answers from the question bank of a Big Data course, together with the unit notes of that course (Units III and IV, covering Apache Pig, Flume, Hive and Sqoop). The adaptive-quiz evaluation uses simulated students instead of real quiz logs. Table 4.1 summarises the data, Table 4.2 lists every file of the AI cohort and Table 4.3 shows sample evaluation records.

Table 4.1 Dataset summary

Attribute	Details
Course corpus (demonstration and verification)	AI cohort, Year 3: 19 files (5 PDF, 6 DOCX, 8 PPTX) for 5 subjects; 801 pages or slides; 893 chunks; 371 image-only pages or slides read by Claude vision; 189 topics
Evaluation corpus	Big Data course notes, Units III and IV (2 PDFs, 48 pages, 48 chunks), indexed in a separate evaluation store
Evaluation questions	60 short examination questions with model answers from the course question bank (30 Unit III, 30 Unit IV); the question bank itself is not indexed
Licence	Internal course material used for academic evaluation only; not redistributed with the code (the notes folder is excluded from the repository)
Target / classes	Each answer is graded as correct, partial or wrong; a “not found” reply counts as wrong
Class distribution	Balanced by unit: each half has 15 Unit III and 15 Unit IV questions
Development / held-out split	30 / 30 (odd- / even-numbered questions); no training split, because nothing is trained
Off-syllabus probe set	15 general questions outside the notes, used only to calibrate abstention (Section 6.2)
Simulated students (quiz study)	200 per mode; 20 topics, 5 genuinely weak (correct-answer probability 25–45 %, others 70–95 %); 10 quizzes of 10 questions; seeds 0–199

Table 4.2 lists the nineteen files of the AI cohort with the subject and unit under which they were shared, the number of pages or slides, the number of image-only pages or slides whose text was recovered by the vision model, the number of chunks stored and the number of topics extracted. The original file names gave little information, for example “UNIT III (1).pptx”, so every file was opened and its subject and unit were

determined from its content before it was renamed and shared; laboratory programs and viva questions were filed as supporting material rather than as numbered units. The table shows how much of the material was unreadable without the image-reading stage: in NLP Units 3, 4 and 5 more than three quarters of the pages contained no extractable text, and the slide decks of Deep Learning Unit 1 and Big Data Unit 2 contained dozens of picture-only slides.

Table 4.2 Files of the AI cohort and what the ingestion pipeline recovered from them

File (subject · name)	Unit	Pages / slides	Image pages read	Chunks	Topics
NLP · Unit 1	Unit 1	67	28	71	8
NLP · Unit 2	Unit 2	44	28	46	11
NLP · Unit 3	Unit 3	73	65	74	8
NLP · Unit 4	Unit 4	89	84	90	13
NLP · Unit 5	Unit 5	97	89	99	12
NLP · Lab Programs - N-grams	Lab programs	3	0	8	8
NLP · Lab Programs - Regular Expressions	Lab programs	2	0	4	7
NLP · Viva Questions (Units 1-5)	Viva questions	11	1	39	12
Deep Learning · Unit 1	Unit 1	141	29	141	12
Deep Learning · Unit 2	Unit 2	48	8	50	11
Deep Learning · Unit 3	Unit 3	41	6	41	10
Deep Learning · Lab - TensorFlow, Keras and PyTorch Basics	Lab programs	15	0	15	8
Big Data · Unit 2	Unit 2	47	26	47	7
Big Data · Unit 3	Unit 3	68	6	68	7
Cyber Security · Unit 3 (Notes)	Unit 3	7	0	21	10
Cyber Security · Unit 3 (Slides)	Unit 3	14	0	15	10
Cyber Security · Unit 4	Unit 4	19	1	19	11
Negotiation Skills · Unit 1	Unit 1	7	0	21	12
Negotiation Skills · Unit 2	Unit 2	8	0	24	12
Total (19 files)		**801**	**371**	**893**	**189**

Table 4.3 Sample records from the evaluation question set (reference answers shortened)

ID	Unit	Question	Reference answer (excerpt)
1	III	What is Apache Pig?	A high-level platform for processing large datasets, designed to work with Hadoop; uses Pig Latin instead of SQL.
2	III	What is Pig Latin?	A high-level scripting language for Hadoop following a data-flow model; the output of one operation is the input to the next.
31	III	How is Pig different from traditional databases?	Pig is used for large-scale batch processing, while databases handle transactional data; Pig uses Pig Latin, not SQL.
56	IV	What is Bucketing in Hive?	Further divides a table or partition into a fixed number of files; improves joins and sampling; created with CLUSTERED BY.

4.3 Pre-processing

Fig. 4.2 shows the ingestion pipeline. Text is extracted page by page from PDF files with pypdf and slide by slide from PowerPoint files with python-pptx, which also walks grouped shapes and tables so that no text box is missed; old .ppt files are first converted with LibreOffice. Word and text files, which have no pages, are split into sections of about four hundred words. When a PDF page is extracted with every word on its own line, which happened with one of the evaluation PDFs, the page is re-extracted in pypdf's layout mode. Lines among the first or last three lines of a page that repeat on at least half of the pages, ignoring digits, are removed as headers and footers. Restricting this rule to page edges was necessary, because a first version also removed the labels "Answer:" and "Example:", which repeat inside the body of a question bank, and the rule is run again after the image-reading stage so that repeated headers in recovered text are also removed.

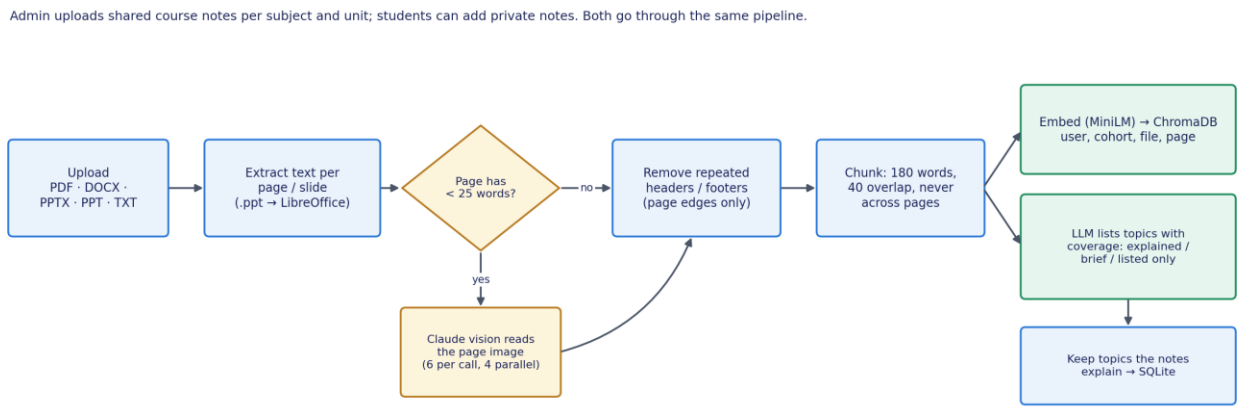


Fig. 4.2 Document ingestion and pre-processing pipeline, including the image-reading stage

A page or slide with fewer than twenty-five extractable words is treated as an image page. For PDFs, such pages are copied into small sub-documents of six pages, with a note giving their original page numbers; for slides, the pictures on six slides are downscaled to at most 1,568 pixels and sent together, skipping images smaller than eight kilobytes such as icons and logos. Each batch is sent to Claude with a JSON schema that maps every original page number to its transcribed text, and four batches run in parallel with a timeout of 180 seconds. Pages are then split into chunks of 180 words with a 40-word overlap, never crossing a page boundary, so every chunk has exactly one page to cite. The chunk size follows from the embedding model, which reads only 256 word-pieces, about 190 English words. Finally the LLM lists the topics of each document and marks each as explained, brief or listed only; topics that the notes

merely list, for example in a syllabus outline, are dropped, so quizzes are never set on them.

4.4 Development / Held-out Split and Avoiding Data Leakage

Because no model is trained there is no training set, and the risk of leakage is instead that the system could see the answers, or that design choices could be tuned to the questions used for the final result. Several measures address this risk. The evaluation questions and reference answers come from the question-bank PDF, and only the two unit PDFs were indexed for retrieval, so the system could not copy the reference answers. The sixty questions were split into odd-numbered development and even-numbered held-out halves, each balanced by unit; the development half was used to find problems and to try changes such as keyword search, and the held-out half was run on the final pipeline only. The chunk size, re-ranker threshold and other retrieval parameters were fixed during feature development, before the evaluation set was built, and every answer and judgement in a run used one pinned model with automatic fallback disabled, so that results were never mixed across models.

Grading was blind: the judge saw the notes-grounded answer and the answer without notes as “Answer 1” and “Answer 2” in random order. The evaluation was run twice on the same questions, first with Gemini Flash-Lite as both the answering model and the judge, when Gemini was the main model of the system, and again with Claude Sonnet 5.5 after Claude became the main model, so that the two runs can be compared on identical questions and an identical index. The remaining risk, that the judge is the same model that wrote the answers, is discussed in Section 6.6. For the AI cohort there is no question bank with reference answers, so that corpus was used for an end-to-end verification rather than an accuracy score: the questions were written before the run, the expected source file of each was fixed in advance, and the check passed only if the answer cited that file.

4.5 System Architecture and Model Choice

PersonaAI follows a layered architecture, shown in Fig. 4.3. The presentation layer is a Streamlit application: app.py shows the sign-in card and the first-sign-in form and routes between the student pages and the administration page in views/, using shared styling in ui/, while scripts/ holds the daily reminder job and the demonstration set-up script. The application layer, core/, contains all the logic and no user-interface code, so

it can be tested directly; it is divided into modules for accounts, ingestion, image reading, retrieval and answering, quizzes, the learner model, the planner, model access, and email and calendar output. The data layer stores application data in SQLite and chunk embeddings in ChromaDB. The model layer consists of two small local models for embedding and re-ranking, the generative models, which are reached only through core/llm.py, and the email service.

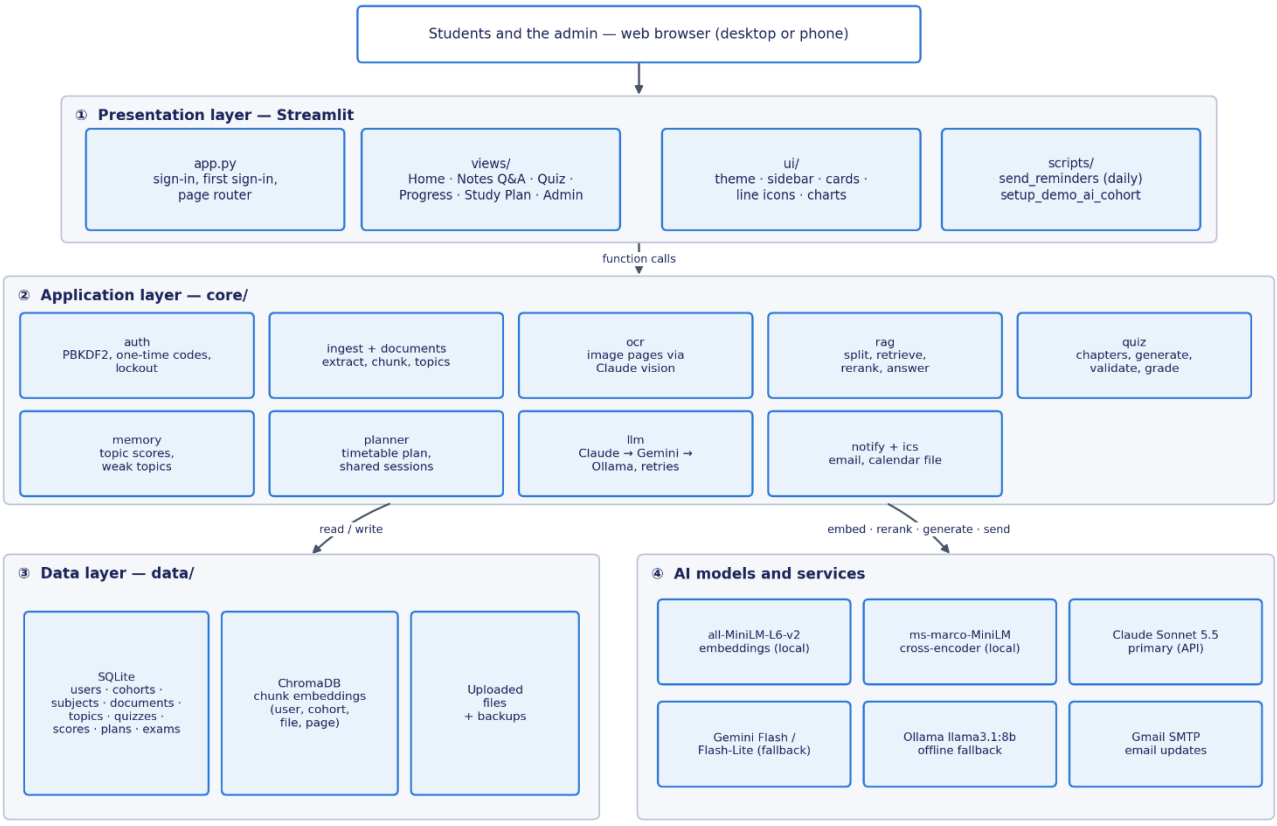


Fig. 4.3 Layered system architecture of PersonaAI

Fig. 4.4 shows how these components interact when a student asks a question, including the splitting of multi-part questions, the threshold that can end the request without calling any model, the provider chain and the NOT_FOUND reply. The choice of each model is justified in Table 4.4. In every case the smallest component that met the requirement was chosen, because the system must run on an ordinary laptop for a whole class, and larger alternatives were kept for comparison in Section 4.8. Claude Sonnet 5.5 replaced Gemini as the primary generator during the project after the free Gemini tier failed frequently with overload and quota errors in daily use; Gemini and the local Llama model remain as fallbacks, so that a failure of the primary provider slows the

system down for a few seconds instead of stopping it, and every failure is logged with its status code.

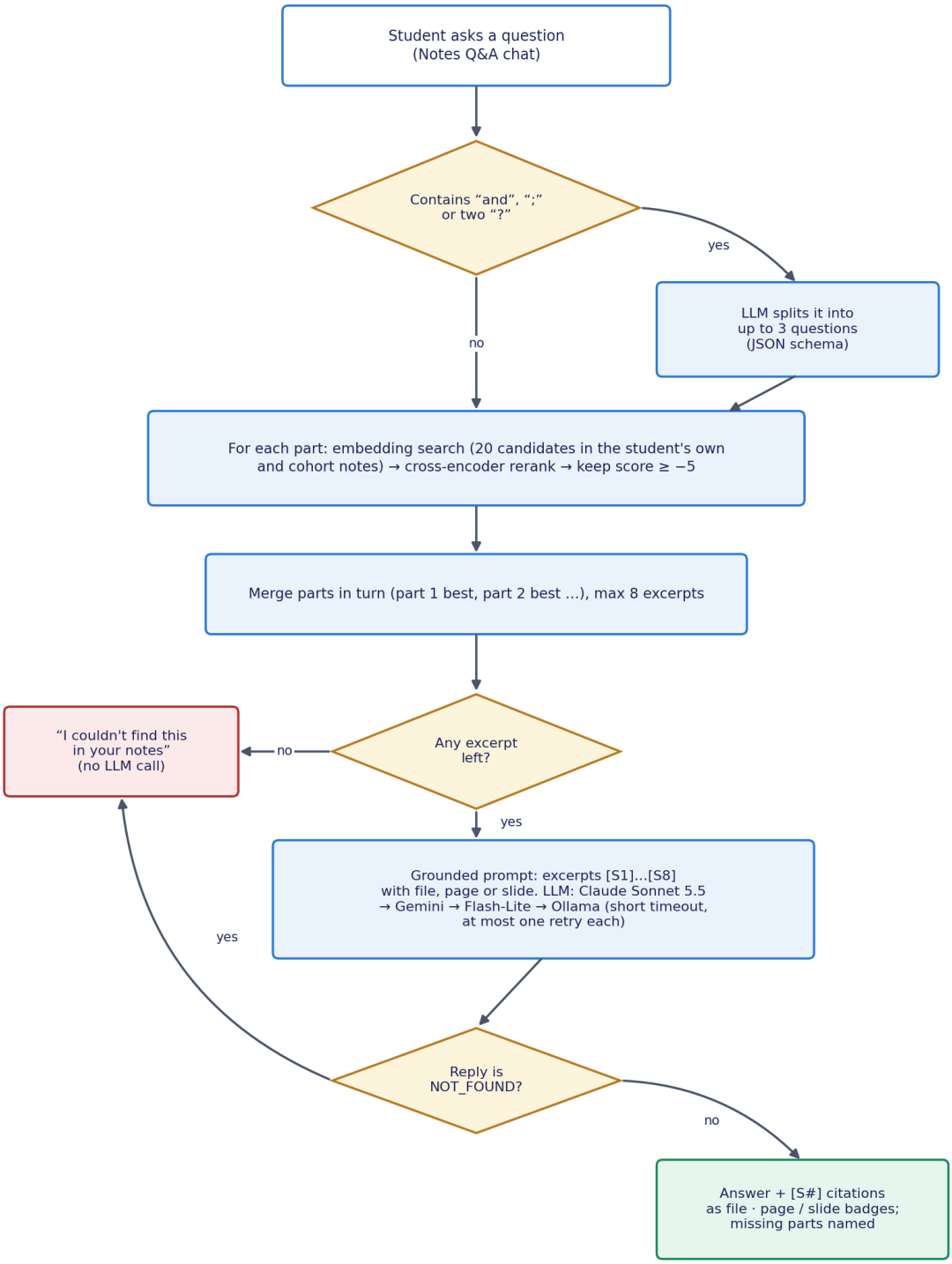


Fig. 4.4 Flow of a question through the system, with the multi-part split and the provider chain

Table 4.4 Model choices and justification

Component	Model used	Justification
Embedding (stage 1)	all-MiniLM-L6-v2 (384-d)	Small and fast on a CPU; strong general-purpose semantic search [12], [13]; runs locally, so notes are not sent anywhere for indexing.
Re-ranker (stage 2)	cross-encoder/ms-marco-MiniLM-L-6-v2	Reads question and passage together, which fixed cases where embedding search ranked the defining passage low [14].
Generator (primary)	Claude Sonnet 5.5	Reliable instruction following, structured JSON output with schemas, and image input for scanned pages [24].
Generator (fallback)	Gemini 3.8 Flash, then 3.5 Flash-Lite [23]	Free tier; keeps the system working when the primary provider fails.
Generator (offline)	Llama 3.1 8B via Ollama [35]	Open-weight model that runs on the laptop GPU; usable without internet or quota.
Image reading	Claude Sonnet 5.5 (vision)	Reads photos, diagrams and screenshots of slides better than classical OCR [22]; returns page-numbered JSON.
Learner model	Accuracy over the last 10 answers per topic	Explainable to students, needs no training data and adapts quickly when a student improves.
Planner	Rule-based weighted allocation	Guarantees full coverage, spacing and exam-date rules, uses no API calls, and is testable.

4.6 Methodology

This section describes the method used for each objective of Section 1.4, from the answering of a question to the security of the accounts. Table 4.5 relates each method to the objective it serves and to the evidence used to judge it; the detailed result for each objective is given in Table 7.1. The methods share two principles. First, the language model is used only where language understanding is needed, namely to read images, to split questions, to write answers and questions and to list topics, while everything that must hold every time, such as access control, scoring, allocation and planning, is done by rules in ordinary code. Second, every step that could fail silently, such as a missing page, an empty retrieval or a malformed question, has an explicit check whose outcome is shown to the student or recorded in the log.

Table 4.5 Objective-to-methodology map

Obj.	Method (section)	Evidence of achievement
O1	Ingestion, image reading and topic coverage (4.3)	Table 4.2; ingestion and OCR tests; browser check
O2	Grounded answering, multi-part split, partial answers (4.6.1–4.6.2)	Held-out accuracy (6.3); browser check (6.4); calibration (6.2)
O3	Quiz generation and validation (4.6.3)	Chapter quiz audit (6.4); validation tests
O4	Weak-topic memory (4.6.4)	Unit tests; demonstration screenshots (6.9)
O5	Adaptive quiz allocation (4.6.5)	200-student simulation (6.5)

Obj.	Method (section)	Evidence of achievement
O6	Timetable planner, calendar and email (4.6.6)	Planner tests; demo-cohort plan; email and calendar tests
O7	Cohorts and accounts (4.6.7)	Cohort and auth tests; isolation check in the browser
O8	Evaluation design (4.4, 4.7, 6.1)	Tables 6.1–6.10
O9	Architecture, fallback and interface (4.5, 5.4)	Fallback tests; logs; screenshots

4.6.1 Grounded Question Answering with Abstention

For a question q, embedding search returns the twenty chunks with the highest cosine similarity between the question embedding e(q) and the chunk embedding e(c), restricted to the documents that the student may see, namely their own private documents and the documents shared with their cohort. The similarity is given by Equation (4.1), where the dot denotes the inner product of the two vectors and the vertical bars denote their lengths. The cross-encoder then gives each candidate a relevance score s(q, c), and the answer is generated from the set S of at most five candidates with the highest scores that also pass the threshold τ, as written in Equation (4.2). The threshold of −5 was calibrated on course notes during development, and its behaviour on on-syllabus and off-syllabus questions is shown in Section 6.2. If the set S is empty, the system replies that the notes do not contain the answer without calling any language model, which saves time and API cost for questions that are clearly outside the course.

$$\mathrm{sim}(q, c) = e(q) \cdot e(c) \,/\, (|e(q)| \times |e(c)|) \tag{4.1}$$

$$S = \text{top-5}\ \{\, c \in C_{20} : s(q, c) \geq \tau \,\}, \quad \tau = -5 \tag{4.2}$$

Otherwise the selected excerpts are numbered [S1], [S2] and so on, each labelled with its file and page or slide, and the LLM is instructed to answer only from them, to cite them inline after the sentence they support, to write in the style of an examination answer with a direct answer first and key points afterwards, and to reply with the single token NOT_FOUND if the excerpts answer none of the question. If they answer only part of it, the model must answer that part and then say plainly which part the notes do not cover, without filling the gap from its own knowledge. Near-domain questions can pass the threshold, so this second, model-level check is needed. The citation tags are parsed from the reply and shown under the answer as badges such as “Deep Learning - Unit 2.pptx, slide 10”, and the cited excerpts can be expanded under the answer, so the student can check every claim against the notes in a few seconds.

4.6.2 Multi-Part Questions

A question such as “What does the YARN Resource Manager do, and what does a pooling layer do in a CNN?” asks about two subjects at once. In the first version of the system a single search was run for the whole question, the Big Data passages took every slot, and the answer claimed that the notes did not explain pooling. The method now has three steps. A cheap regular expression first selects questions that might have several parts, namely those containing the word “and”, a semicolon or two question marks, so single questions never pay for an extra model call. The LLM then splits such a question into at most three standalone questions, each repeating the shared context, returned in a fixed JSON schema. Each part is retrieved and re-ranked on its own, and the results are merged by taking the best passage of every part in turn, then the second best of every part, and so on, up to eight excerpts, so that every part is represented near the start of the context, in line with the findings of Liu et al. [18].

4.6.3 Quiz Generation and Validation

A quiz is described by an allocation that maps each topic to a number of questions. The student first chooses a subject and then one or more chapters, where a chapter is a subject and unit, for example “Deep Learning · Unit 2”, so that the unit’s notes and slides count as one chapter. The best excerpts for each chosen topic are retrieved, and a single LLM call asks for exactly the allocated number of questions per topic, using a structured-output schema in which the topic tag and the source tag are enumerations of the allowed values, each question has exactly four options and an answer letter, and an explanation is required. Every question is then validated by rules, missing questions are requested again up to three calls per quiz, options are shuffled so that the correct answer is not always where the model tends to put it, and for the weaker local model an additional check asks the model to answer its own questions without the key.

4.6.4 Weak-Topic Memory

Let R_t be the most recent answers of the student on topic t, up to ten of them. The accuracy of the topic is defined by Equation (4.3), where correct(a) is one for a correct answer and zero otherwise and $|R_t|$ is the number of answers in R_t. The status of the topic is “not tested” if the student has no answers on it, “learning” if there are fewer than three answers, “weak” if the accuracy is below 0.60, “strong” if it is at least 0.80, and “learning” otherwise. Using recent rather than all-time accuracy lets the status change when the student improves: a student who answered a topic wrongly ten times

and then correctly nine times would be at 47 per cent all-time, which is weak, but at 90 per cent over the last ten answers, which is strong. Topic scores are not updated incrementally; after every quiz they are rebuilt from the saved answers in one SQL statement, so they can never drift from the underlying data, and the same rule is applied to every student of every cohort.

$$\mathrm{acc}(t) = (1 / |R_t|) \cdot \Sigma\ \mathrm{correct}(a) \ \text{ over } a \in R_t, \quad |R_t| \leq 10 \tag{4.3}$$

4.6.5 Adaptive Quiz Allocation

For an adaptive quiz of n questions, round(0.6 n) questions go to weak topics, weakest first, cycling through them if there are fewer weak topics than questions. The remaining questions go first to untested topics, in order to discover new weak spots, then to the lowest-scoring learning topics, and finally to strong topics that were seen longest ago, which gives a light spaced-review effect. If no topic is weak yet, all questions follow the second rule. The even-mix mode spreads the questions as evenly as possible over randomly ordered topics and serves as the baseline in Section 6.5. On the Notes Q&A page, the topic of a question is also matched against the student's weak topics by embedding similarity, and when the question concerns a weak topic the page says so and suggests a short quiz, so that the learner model is visible wherever the student is working rather than only on the Progress page.

4.6.6 Timetable Study Planner, Calendar and Email

The planner is rule-based rather than generated by an LLM, so that its guarantees always hold and it makes no API calls. The student enters an examination timetable with a subject, a date and an optional start time for each paper, and the number of study hours per day. Every topic receives a weight w of three if it is weak, two if it is untested or learning and one if it is strong. Each day is divided into thirty-minute slots and shared between the subjects whose examination is still ahead, in proportion to the subject's total weight divided by the number of days left, as in Equation (4.4), so that the nearest and weakest subjects receive the largest share; shares are rounded with the largest-remainder method so that every slot of the day is used. The day before each examination is that subject's revision day, an examination day keeps half a day for the remaining subjects, and nothing of a subject is planned after its examination. Fig. 4.5 summarises the planner.

$$\text{share}(s, d) = W(s) / (\text{date}(s) - d) \div \Sigma \text{ over subjects } u \text{ ahead of } d \text{ of } W(u) / (\text{date}(u) - d) \quad (4.4)$$

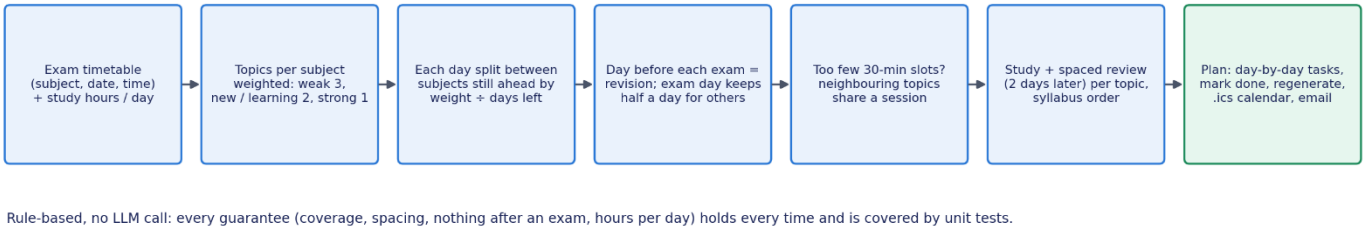


Fig. 4.5 Rule-based timetable planner

Within a subject, the slots are divided between topics in proportion to their weights with the largest-remainder method, and each topic's time is split into a study session of about sixty per cent and a review session with a five-question quiz of about forty per cent at least two days later, applying the spacing effect [5]. With the 189 topics of the AI cohort there is often too little time for a separate session per topic, so neighbouring topics in syllabus order then share a session, which is weighted like its weakest member and named after all of them, and the plan says how many topics were grouped. A finished plan can be downloaded as an iCalendar file [36], in which each study day is an event ending at 22:00 with a reminder thirty minutes before and each examination is an event with reminders a day and an hour before, and a daily script emails each student who asked for it the tasks of the day and the next examination.

4.6.7 Cohorts, Accounts and Security

An administrator imports a class list in CSV format with the hall ticket number, name, department and year of each student; every distinct department and year becomes a cohort, and re-importing the list updates names and cohorts without touching passwords or history. The administrator adds the subjects of a cohort and shares notes per subject and unit, and every query for a student is restricted, both in SQL and in the vector index, to the student's own documents and those shared with their cohort. Students have no password at first; the administrator issues one-time sign-up codes, which are shown once and stored only as hashes. On the first sign-in the student enters the code, an email address and a new password, which is stored as a salted PBKDF2 hash with 200,000

iterations [37], and the code is used up. Following common guidance on authentication [38], five failed sign-ins within five minutes lock the account for five minutes, as shown in Fig. 4.6, and the administrator can reset a forgotten password by issuing a new code.

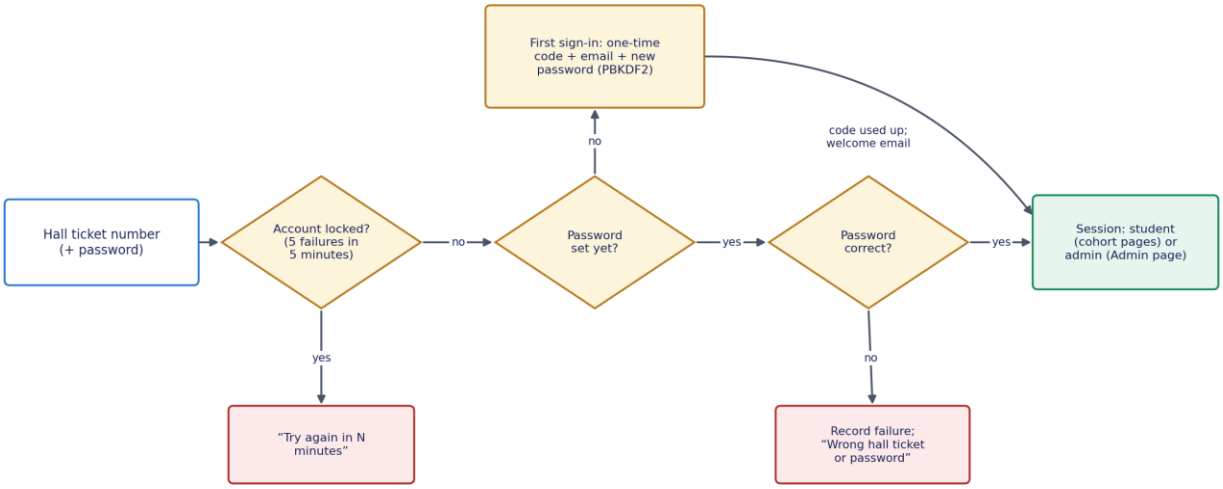


Fig. 4.6 Sign-in flow with the first-sign-in form and the lockout

The primary actors are the student and the department administrator; the LLM service and the email service are secondary actors. The student signs in, asks questions, uploads private notes, takes quizzes, views progress, creates a timetable plan, adds it to a calendar and chooses email updates, while the administrator imports class lists, issues codes and shares notes. Application data is stored in eleven SQLite tables, described in Table 4.6; foreign keys cascade on delete, so removing a document removes its topics, and topic scores are derived from the saved answers. Chunk text and embeddings are stored in ChromaDB with metadata giving the user, the cohort, the document, the file name and the page, which is used both for filtering and for building citations, so the vector index and the relational database always agree on who may see what.

Table 4.6 Database tables

Table	Purpose	Key columns
cohorts	A department and year	id, department, year
users	Students and their accounts	id, username (hall ticket), password_hash, cohort_id, setup_code_hash, email, email flags
login_failures	Failed sign-ins for the lockout	username, failed_at
subjects	Subjects of a cohort	id, cohort_id, name
documents	Private and shared notes	id, user_id, cohort_id, subject_id, filename, unit, page_count
topics	Topics per document	id, user_id, document_id, name
quiz_attempts	One row per submitted quiz	id, user_id, created_at, score, total

Table	Purpose	Key columns
quiz_answers	Every answer with its topic	attempt_id, topic_id, question, chosen, correct, is_correct
topic_scores	Derived accuracy per topic	user_id, topic_id, attempts, correct, accuracy, last_seen
study_plans	Saved plans (JSON)	id, user_id, exam_date, hours_per_day, plan_json
exams	A student's examination timetable	id, user_id, subject, exam_date, start_time

4.7 Baseline Models for Comparison

Each design decision that the evaluation tests is compared against a simpler baseline, listed in Table 4.7. The most important is the first baseline, in which the same language model answers the examination questions from its general knowledge with the same examination-style instructions but without any notes; it measures what retrieval adds or costs on these questions. The other baselines isolate individual components: a hybrid first stage that adds BM25 keyword search [11], a larger re-ranker, an even spread of quiz questions instead of the adaptive allocation, all-time accuracy instead of the recent-answer window, a single search instead of the multi-part split, and ingestion without the image-reading stage. Where a baseline could not be run as a controlled experiment, the evidence was taken from a specific observed case during development and is reported as such in Section 6.7, so that observed cases are never presented as if they were measured averages.

Table 4.7 Baselines used in the evaluation

Baseline	What it is	Compared with	Where
B1: LLM without notes	The same model answers from general knowledge with the same exam-style instructions	Full RAG pipeline	6.3, 6.5
B2: Hybrid first stage	Embedding search plus BM25 keyword search, then the same re-ranker	Embedding-only first stage	6.7
B3: Larger re-ranker	ms-marco-MiniLM-L-12-v2 instead of L-6 (retrieval-only check)	L-6 re-ranker	6.7
B4: Even-mix quizzes	Questions spread evenly over all topics	Adaptive allocation	6.5
B5: All-time accuracy	Topic accuracy over all answers	Last-ten-answers window	4.6.4, 6.7
B6: Single search	One retrieval for a multi-part question	Per-part retrieval and merge	6.7
B7: No image reading	Text extraction only	Extraction + vision model	6.4, 6.7

4.8 Comparison with Recent (State-of-the-Art) Models

Table 4.8 places each component next to recent and stronger alternatives and explains why the simpler choice was made. Only the re-ranker alternative and the two generators were measured in this project; the others are listed for context and as future work. The general pattern is that the components that run for every passage of every question, namely the embedding model and the re-ranker, were kept small because they run on the laptop for the whole class, while the generator, which runs once per question and is hosted by its provider, was chosen for quality and reliability. The error analysis in Section 6.6 supports this split: almost every incomplete answer was caused by the way a passage was cut from its section, not by the quality of the embeddings or by the generator, so a larger embedding model would not by itself have fixed the observed failures.

Table 4.8 Components used versus recent alternatives

Component	Used in PersonaAI	Recent alternatives	Reason for the choice
Retrieval embeddings	all-MiniLM-L6-v2	E5 [16], BGE [17]	Larger and slower on a CPU; misses were mostly caused by chunk context (6.6).
Re-ranker	MiniLM-L-6 cross-encoder	MiniLM-L-12 (tested), larger re-rankers	L-12 found the answer for one more question in 30, at higher memory and latency.
RAG design	Retrieve, re-rank, generate, with question splitting	Self-RAG [9], modular RAG [8]	Self-RAG needs a fine-tuned generator; not possible with a hosted API model.
Generator	Claude Sonnet 5.5; Gemini; Llama 3.1 8B	Larger frontier and open-weight models	Measured on the same questions with two providers (6.3).
Image reading	Claude vision on image-only pages	Classical OCR [22]	Handles photos, diagrams and slide screenshots; returns page-numbered text.
Learner model	Recent-accuracy thresholds	BKT [26], DKT [27], HLR [28]	Need many answers per student; less explainable.
RAG evaluation	Blind LLM judge against reference answers	RAGAs metrics [20]	Reference answers were available, so correctness could be graded directly.

CHAPTER 5
IMPLEMENTATION

5.1 Training Setup (Inference and Configuration Setup)

PersonaAI uses pretrained models only, so the usual training settings, namely the optimizer, learning rate, loss function, number of epochs, batch size and early stopping, do not apply to it. Table 5.1 lists, in the format of the department's training-setup table in Annexure IX, the settings that do determine the behaviour and the results of the system: the models and their order, the retrieval and chunking parameters, the timeouts and retry policy, the image-reading parameters, the evaluation model and the random seeds of the simulation. All settings are read from a git-ignored .env file or from constants at the top of each module, so the configuration used for every result in Chapter 6 can be read from the code and the environment without guesswork, and changing a provider or a model name does not require any change to the code.

Table 5.1 Inference and configuration setup

Parameter	Value
Model / backbone	all-MiniLM-L6-v2 (embedding), ms-marco-MiniLM-L-6-v2 (re-ranker), claude-sonnet-5-5 → gemini-3.8-flash → gemini-3.5-flash-lite → llama3.1:8b (generation); all pretrained, none fine-tuned
Optimizer, learning rate, loss, epochs, batch size	Not applicable: no training or fine-tuning was performed
Retrieval	20 candidates → re-rank → top 5 (up to 8 for multi-part questions); threshold −5; cosine distance
Chunking	180 words, 40-word overlap, minimum 30 words, never across pages
Timeouts and retries	Claude 45 s, Gemini 30 s, Ollama 120 s; at most one retry; a provider that timed out is skipped for 60 s
Image reading	Pages under 25 words; 6 pages or slides per call; 4 calls in parallel; 180 s timeout; images ≤ 1,568 px
Generation settings	No temperature for Claude (fixed by the API); 0.2 for Gemini answers, 0.7 for quiz writing, 0 for the judge
Evaluation model	Run 1: gemini-3.5-flash-lite; run 2: claude-sonnet-5-5; one pinned model per run, fallback disabled
Random seed	Simulation: students seeded 0–199 per mode; quiz option shuffling in the application is intentionally unseeded
Hardware and run time	AMD Ryzen 7 7445HS, 16 GB RAM, RTX 4050 Laptop GPU; indexing the AI cohort ≈ 335 s including image reading
Framework and version	Python 3.12.10, PyTorch 2.14.1, sentence-transformers 6.1.0, ChromaDB 1.5.9, Streamlit 1.65.0, anthropic 2.3.0

5.2 Hyper-parameter Tuning Approach

There are no learned hyper-parameters in the system; instead the pipeline has design parameters, listed with their values in Table 5.2. Each was either derived from a constraint, such as the input limit of the embedding model or the context length of the local model, or chosen by testing on course notes and on the development questions, never on the held-out questions. Parameters that control how the system treats students, such as the weak and strong thresholds and the share of adaptive questions on weak topics, are design choices that were made explicit and covered by tests rather than tuned to a metric, because no data on real students existed to tune them against. The image-reading threshold of twenty-five words was chosen by inspecting the AI cohort's pages: title slides and section dividers have fewer words but still carry their text, so a higher threshold would have sent many readable slides to the vision model at extra cost.

Table 5.2 How the design parameters were chosen

Parameter	Value	How it was chosen
Chunk size / overlap	180 / 40 words	Derived from the embedding model's 256-token input limit
Candidates / top-k	20 / 5 (8 multi-part)	Re-ranker cost on a CPU; few, well-ranked passages suit the LLM [18]
Re-ranker threshold	−5	Calibrated on on-syllabus versus unrelated questions (Section 6.2)
First-stage retrieval	Embedding only	BM25 hybrid tried on the development set; no gain, reverted (6.7)
Image-page threshold	< 25 words	Inspection of the AI cohort's pages and slides
Topic coverage filter	Keep explained; brief only if < 3 explained	Inspection of extracted topics on syllabus pages
Recent-answer window	10 answers	Lets a status change after a few correct answers
Weak / strong thresholds	< 60 % / ≥ 80 %	Common pass and distinction bands; minimum 3 answers
Adaptive weak share	60 %	Majority on weak topics while still exploring others
Planner weights	3 / 2 / 1	Tests check that weak > learning > strong time
Lockout	5 failures / 5 minutes	Common authentication guidance [38]

5.3 Key Code Snippets

This section explains the most important parts of the code. The listings are taken directly from the project source at the time the report is built, so they always match the submitted code; the complete code is in the GitHub repository given in Appendix A, and its structure is described in Appendix C. The listings were chosen to show one

example of each kind of logic in the system: the retrieval rules, the use of the language model with a fixed output structure, the handling of image-only pages, the provider chain with its retry policy, the rules that validate generated questions, the learner model, the planner's handling of short timetables, and the account lockout. Each listing is preceded by a short explanation of why it is written the way it is and which problem observed during development it solves, because most of these functions were changed at least once after testing on the real course notes.

5.3.1 Two-Stage Retrieval and Multi-Part Questions

Listing 5.1 implements Equations (4.1) and (4.2). The re-ranker was added after testing on real notes: for the question "What is Apache Pig?", embedding search ranked the defining chunk eighth, because almost every chunk of the unit was about Pig, and the answer came back as not found, while the cross-encoder ranked the same chunk first with a score of 7.4 against 1.9 for the next one. Listing 5.2 shows the multi-part handling of Section 4.6.2: sub_questions sends only candidate questions to the model and falls back to the original question if the reply cannot be parsed, and retrieve_for_answer merges the parts in turn. The fallback matters, because a failure of the splitting call must never prevent an answer; in the worst case the system behaves exactly as it did before the feature was added, with one search for the whole question.

Listing 5.1 Two-stage retrieval (core/rag.py)

```
def retrieve(
    user_id: int, question: str, document_ids: list[int] | None =
None, k: int = TOP_K
) -> list[vectorstore.Hit]:
    """Best k relevant chunks for the question, best first (empty
if none are relevant)."""
    candidates = vectorstore.search(user_id, question,
k=CANDIDATES, document_ids=document_ids)
    if not candidates:
        return []
    scores = _cross_encoder().predict([(question, h.text) for h in
candidates])
    for hit, score in zip(candidates, scores):
        hit.score = float(score)
    ranked = sorted(candidates, key=lambda h: h.score,
reverse=True)
    return [h for h in ranked if h.score >= MIN_SCORE][:k]
```

Listing 5.2 Splitting multi-part questions and merging their excerpts (core/rag.py)

```
def sub_questions(question: str) -> list[str]:
    """The separate questions inside a multi-part question ("What
does YARN's Resource
    Manager do, and what does a pooling layer do?"); [question] if
it asks one thing.
    Searching once for the whole question lets one part take every
excerpt slot."""
    if not _MAYBE_MULTI.search(question):
        return [question]
    try:
        reply = llm.generate(SPLIT_PROMPT.format(question=question,
n=MAX_PARTS), json_mode=True,
                             schema=SPLIT_SCHEMA, temperature=0)
        parts = [p.strip() for p in json.loads(reply)["questions"]
if isinstance(p, str) and p.strip()]
    except (llm.LLMError, ValueError, KeyError, TypeError):
        return [question]
    return parts[:MAX_PARTS] if len(parts) > 1 else [question]


def retrieve_for_answer(
    user_id: int, question: str, document_ids: list[int] | None =
None
) -> list[vectorstore.Hit]:
    """retrieve(), but a multi-part question gets the best excerpts
for each part, taken in
    turn (part 1's best, part 2's best, ...) so every part is
represented."""
    parts = sub_questions(question)
    if len(parts) == 1:
        return retrieve(user_id, question, document_ids)
    ranked = [retrieve(user_id, part, document_ids) for part in
parts]
    merged, seen = [], set()
    for i in range(TOP_K):
        for hits in ranked:
            if i < len(hits) and (hits[i].document_id,
hits[i].text[:80]) not in seen:
                seen.add((hits[i].document_id, hits[i].text[:80]))
                merged.append(hits[i])
    return merged[:MAX_EXCERPTS_MULTI]
```

5.3.2 Grounding Instructions

Listing 5.3 is the system prompt given to the language model for every question. Two details came from observed failures. The excerpt tags were changed from [1] to [S1] after the model confused a question number in the notes, “22. What is bucketing?”, with an excerpt number, and the partial-answer rule was added after a question about striding and padding was refused outright even though the padding slide had been retrieved. With the rule, the answer explains padding with citations and then states that the excerpts do not explain striding, which is honest and still useful to the student. The prompt also fixes the style of the answer, a direct answer first and key points afterwards,

in the way a good examination answer is written, because short, structured answers are quicker to compare with the notes during revision and make every citation easy to check.

Listing 5.3 Grounding instructions given to the model (core/rag.py)

```
SYSTEM_PROMPT = f"""You are PersonaAI, a study assistant for a
university student.
Answer the question using ONLY the numbered excerpts from the
student's own notes.

Rules:
- Use only facts stated in the excerpts. Do not add outside
knowledge.
- Cite the excerpts you used inline with their tags, like [S1] or
[S2][S4], right after the
  sentence they support. Only use these [S#] tags for citations,
never the notes' own numbering.
- If the excerpts answer only part of the question, answer that
part and then say plainly
  which part the notes don't cover. Do not fill the gap from
outside knowledge.
- If the excerpts answer none of it, reply with exactly {NOT_FOUND}
and nothing else.
- Write the way a good exam answer reads: a one-line definition or
direct answer first,
  then key points as bullets. Include a short example or syntax if
the excerpts give one.
- Keep it concise."""
```

5.3.3 Reading Image-Only Pages

The entry point of the image-reading stage, fill_image_pages in core/ocr.py, finds the pages that need reading, groups them into batches of six, builds either a small PDF or a set of slide pictures for each batch, runs up to four batches in parallel, and writes the transcribed text back into the matching pages, counting the tokens used so that the cost of every upload is known. Pages that the model could not read are counted rather than silently dropped, and the count is shown to the administrator after the upload. For the AI cohort this stage read 371 pages and slides, of which 0 out of 18 picture slides in the Cyber Security Unit 3 deck yielded text, because those pictures were decorative illustrations without any words; this case is discussed in the error analysis of Section 6.6, since it shows a real limit of what any text recognition method can recover from slides.

5.3.4 Model Access, Timeouts and Fallback

All generation goes through core/llm.py, which tries the configured primary provider and then the fallbacks in a fixed order (Listing 5.4). Each provider has its own timeout, at most one retry, and the real status code and message of every failure are written to

the application log, which made it possible to diagnose the frequent Gemini overload and quota errors that motivated the switch to Claude. The retry policy works as follows: an overload is retried once after a short pause, a per-minute rate limit with a short retry delay is waited out on the same model, a daily quota puts the model on a cool-down so that later calls skip it, and a timeout marks the provider as slow for a minute. An earlier version treated every rate limit as an outage and fell back to the much slower local model, which made a short burst of questions take minutes instead of seconds.

Listing 5.4 Order in which providers are tried (core/llm.py)

```
def _candidates(primary: str, fallback: bool) -> list[tuple[str,
str]]:
    """(provider, model) pairs to try in order.

    Gemini's free tier often returns 503 "high demand" for one
model while a lighter
    model still answers, so the lighter Gemini model is tried
before slow local Ollama.
    """
    claude = [("claude", config.ANTHROPIC_MODEL)]
    gemini = [("gemini", config.GEMINI_MODEL)]
    if config.GEMINI_FALLBACK_MODEL and
config.GEMINI_FALLBACK_MODEL != config.GEMINI_MODEL:
        gemini.append(("gemini", config.GEMINI_FALLBACK_MODEL))
    ollama = [("ollama", config.OLLAMA_MODEL)]
    if not fallback:
        return {"claude": claude, "gemini": gemini, "ollama":
ollama}[primary][:1]
    return {"claude": claude + gemini + ollama, "gemini": gemini +
ollama,
            "ollama": ollama + gemini}[primary]
```

5.3.5 Quiz Validation

Each generated question must have exactly four distinct, non-empty options, no “All of the above” or “None of the above”, an answer given as a letter or as the exact text of an option, and a known topic tag, as checked by the function _validate in core/quiz.py. Numeric answers are rejected because models mix zero-based and one-based indices, which would silently mark the wrong option as correct. Shuffling the options removes the model’s position bias: over four thousand shuffles, each position held the correct answer between 24 and 26 per cent of the time. With structured outputs from Claude, malformed questions became rare, but the rules are kept because the fallback models do not support the same schema guarantees, and a single invalid question would otherwise reach the student. Questions that fail validation are requested again for the same topic, so the student still receives the number of questions they asked for.

5.3.6 Learner Model and Planner

Listing 5.5 classifies a topic from its statistics according to the rules of Section 4.6.4; the statistics themselves are rebuilt by one SQL statement after every quiz. Listing 5.6 shows how the planner fits more topics than there are sessions. With the 189 topics of the AI cohort, every realistic timetable was at first refused with a message saying there was not enough time, because each topic needed its own thirty-minute session; the function now groups neighbouring topics in syllabus order, weights each group like its weakest member and reports how many topics were grouped, so that a plan is always produced and the student is told how to give each topic its own time. This change was found by trying the planner with the real cohort, which shows why the system was verified on real course data and not only on the small examples used in the unit tests.

Listing 5.5 Classifying a topic as weak, learning or strong (core/memory.py)

```
def classify(accuracy: float | None, answers: int) -> str:
    if not answers or accuracy is None:
        return "new"
    if answers < MIN_ANSWERS:
        return "learning"
    if accuracy < WEAK_BELOW:
        return "weak"
    if accuracy >= STRONG_FROM:
        return "strong"
    return "learning"
```

Listing 5.6 Sharing sessions between neighbouring topics when time is short (core/planner.py)

```
def _group_topics(topics: list[TopicStat], n: int) ->
tuple[list[TopicStat], dict[int, list[int]]]:
    """Fit topics (in syllabus order) into at most n sessions. With
more topics than
    sessions, neighbouring topics share one: it is named after all
of them and weighted
    like its weakest member, whose id it takes (so a weak topic is
still flagged).
    Returns the sessions and {session id: ids of the topics in
it}."""
    if len(topics) <= n:
        return topics, {t.topic_id: [t.topic_id] for t in topics}
    size, extra = divmod(len(topics), n)
    sessions, members, start = [], {}, 0
    for i in range(n):
        part = topics[start:start + size + (i < extra)]
        start += len(part)
        weakest = max(part, key=lambda t: (WEIGHTS[t.status], -
(t.accuracy or 0)))
        sessions.append(replace(weakest, name=", ".join(t.name for
t in part)))
        members[weakest.topic_id] = [t.topic_id for t in part]
    return sessions, members
```

5.3.7 Account Lockout

The sign-in function in core/auth.py refuses a locked account before checking anything else, records every failure with its time, and clears the failure record on success, so that five failed attempts in a row lock the account for five minutes. Attempts with an unregistered hall ticket number are answered with a message asking the student to contact the administrator, and they count towards the lockout as well, which limits guessing. A student who has no password yet is directed to the first-sign-in form, and the administrator password is compared in constant time with the value in the environment file; administrator sign-in is switched off entirely when that value is empty. The clock used by these functions can be replaced in tests, which allowed the lockout and its expiry to be tested in milliseconds rather than by waiting five real minutes, and the same approach was used for the planner and the reminder emails, whose behaviour depends on the current date.

5.4 Deployment and User Interface

5.4.1 Deployment

The application runs with the command streamlit run app.py. Settings, including the API keys, the administrator password and the email account, are read from a git-ignored .env file locally or from Streamlit secrets when hosted, so no secret is stored in the code or the repository, and none is ever written to the log. On first use after the server starts, the embedding and re-ranking models are loaded once and shared by all users, which takes about thirty seconds; later requests reuse them. A separate script, scripts/send_reminders.py, sends the morning study-plan emails and is scheduled once a day with the operating system’s task scheduler, because a Streamlit page runs only while someone has it open. The demonstration cohort is created by scripts/setup_demo_ai_cohort.py, which backs up the database, organises the notes, creates the students and uploads every file.

5.4.2 User Interface

app.py uses Streamlit’s navigation with the page list hidden, and ui/shell.py draws a custom sidebar with the logo, navigation links and a profile card that shows the student’s hall ticket, cohort and the AI model in use (Fig. 5.1). After feedback that the first design looked informal, the interface was redesigned with straight two-pixel corners, a flat navy welcome banner and monochrome line icons instead of colourful emoji. One primary colour is used for actions, and green, amber and red are reserved

for status; status is always shown with a word or icon as well as colour. All styling lives in one module, ui/styles.py; text provided by users, such as file and topic names, is escaped before it is inserted into custom components; and layouts reflow to a single column on phones. Table 5.3 lists the pages of the application with their purpose and main components.

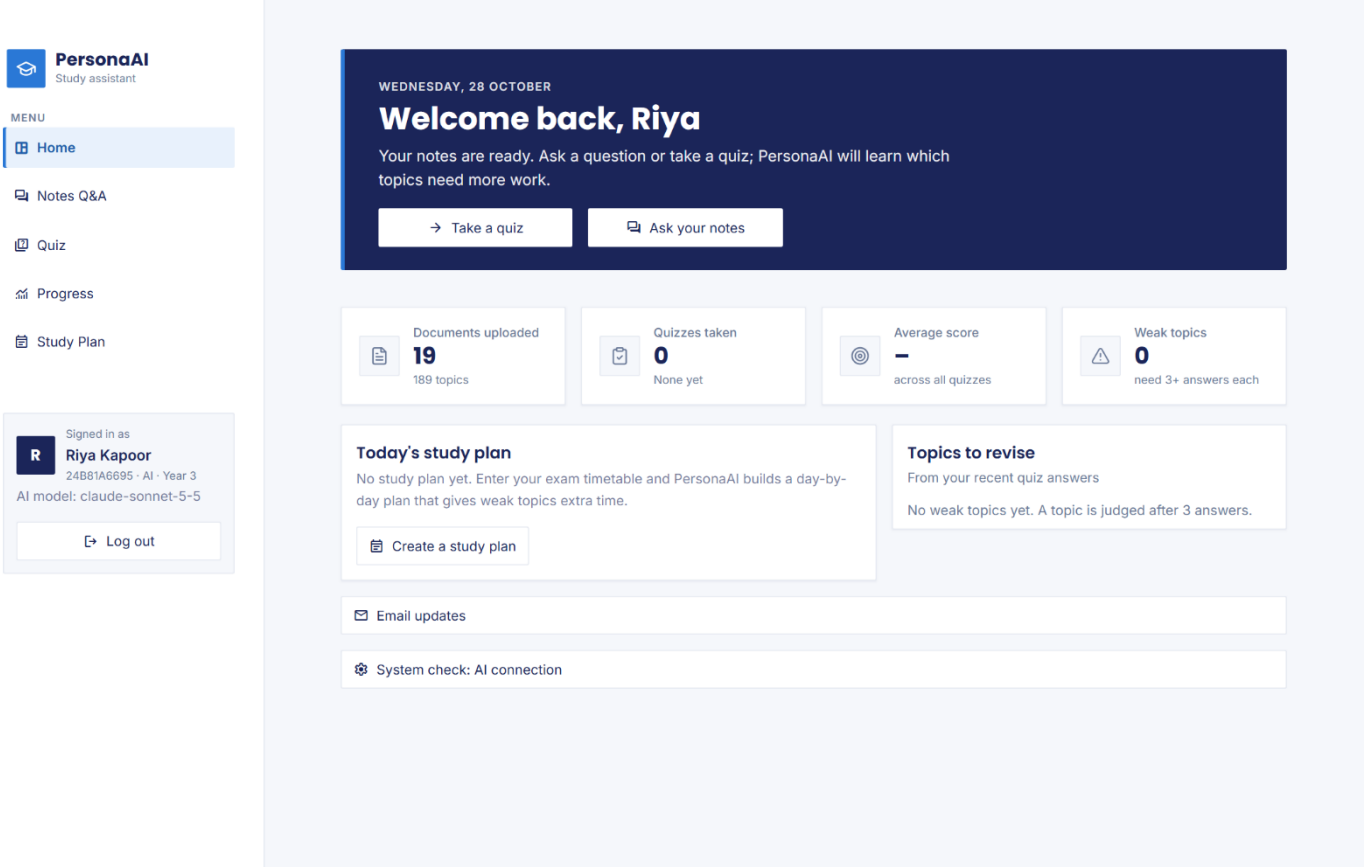


Fig. 5.1 Home dashboard of a new student, with the sidebar navigation and profile card

Table 5.3 Pages of the application

Page	Purpose	Main components
Sign-in	Identify the student or admin	Hall ticket and password; first-sign-in form with code, email and new password
Home	Overview and next step	Welcome banner, four statistic tiles, today's plan, topics to revise, recent quizzes, email updates
Notes Q&A	Ask questions; add private notes	Course notes by subject and unit, suggested questions, chat with citation badges and source excerpts
Quiz	Generate, take and review quizzes	Subject and chapter pickers, size and mode, one question per card, results and review
Progress	Show the learner model	Statistic tiles, weak and strong topics, score trend and per-topic accuracy charts
Study Plan	Plan revision for the timetable	Timetable editor, day-by-day plan, done checkboxes, regeneration, calendar export

Page	Purpose	Main components
Admin	Run the department's cohorts	Class-list import, cohorts, students and codes, subjects, shared notes per unit

5.5 Testing

Testing combined four levels. Unit tests cover pure logic such as chunking, header removal, answer and citation parsing, question splitting, quiz validation, allocation, classification, planner rules, password hashing, lockout, calendar files and email content, with fake model clients and a fake mail server. Integration tests use the real embedding and re-ranking models with a temporary ChromaDB and SQLite database. User-interface tests use Streamlit's AppTest, which runs each page headlessly and drives its widgets. Finally, end-to-end checks use Playwright [39] to drive the running application in a headless Chromium browser with real models on the AI cohort's notes. Table 5.4 summarises the automated suite of 159 tests, all of which pass, Table 5.5 gives sample test cases, and Table 5.6 lists defects that were found and fixed during development.

Table 5.4 Automated test suite (159 tests, all passing)

Test file	Tests	What is covered
test_quiz.py	26	Validation rules, answer formats, retries, answer-key check, schema, chapter picker
test_memory.py	16	Classification, recent-answer window, adaptive allocation, topic matching
test_app.py	15	Sign-in, first sign-in with email, every page, quiz flow, plan, admin
test_llm.py	15	Provider order, timeouts, retries, quota cool-down, logging of failures
test_cohorts.py	14	Cohorts, shared notes, isolation, codes, lockout, password reset
test_planner.py	14	Coverage, weighting, spacing, final days, shared sessions, regeneration
test_documents.py	10	Chunking, page boundaries, header removal, DOCX/TXT extraction
test_rag.py	9	NOT_FOUND, citations, slide labels, question splitting, merging
test_planner_exams.py	9	Timetables, revision days, nothing after an exam, crowded timetables
test_slides_ocr.py	7	PowerPoint extraction, image pages, OCR batching with a fake model
test_eval.py	6	Blind judging order, abstentions, quota stop, simulation
test_db.py / test_ingest.py	10	Schema, cascades, ingest → search → answer → delete, user isolation
test_notify.py / test_ics.py	8	Email content, reminder script, login failure messages, calendar events

Table 5.5 Sample test cases

ID	Test case	Input / action	Expected result	Result
TC1	First sign-in	Code, invalid email, then valid email and password	Email rejected first; then signed in, email saved	Pass
TC2	Lockout	Five wrong passwords	Account locked for 5 minutes	Pass
TC3	Cohort isolation	CSE student lists documents	No AI-cohort notes visible	Pass
TC4	Cited answer	“What is a context-free grammar?”	Answer citing NLP - Unit 3.pdf	Pass
TC5	Off-syllabus question	Kubernetes scheduler question	“I couldn't find this in your notes”	Pass
TC6	Multi-part question	YARN and pooling question	Both subjects cited	Pass
TC7	Image-only page	PDF page with no text, fake vision model	Page text filled in and cited	Pass
TC8	Invalid MCQ	Model returns 3 options	Question dropped and regenerated	Pass
TC9	Improvement	10 wrong then 9 right	Topic becomes strong (90 % recent)	Pass
TC10	Crowded timetable	5 exams, 200 topics, 3 h/day	Plan produced; topics share sessions	Pass
TC11	Calendar export	Plan → .ics	Study and exam events with reminders	Pass
TC12	Provider failure	Claude times out	Gemini answers; failure logged with status	Pass

Table 5.6 Defects found during development and their fixes

Defect	How it was found	Fix
“What is Apache Pig?” answered “not found”	Live Q&A on course PDFs	Cross-encoder re-ranking of 20 candidates
Header removal deleted “Answer:” labels	Inspecting extracted text	Only remove repeats at page edges
Model cited “[22]”, a question number	Live Q&A	Excerpt tags changed to [S1]…[S8]
Gemini failing often with overload and quota errors	Application log	Claude as primary; timeouts; one retry; logging
Most of three NLP units unsearchable	Inspecting the AI cohort’s files	Claude vision for image-only pages (371 pages)
Topics extracted from syllabus lists	Reviewing extracted topics	Coverage field; keep explained topics only
Cross-subject question cited one subject	Browser check	Question splitting and per-part retrieval
Padding question refused although found	Browser check	Partial-answer rule in the prompt
Every realistic timetable refused	Planner dry run on the AI cohort	Neighbouring topics share sessions
Tests could send real email	Code review	Email disabled in the test configuration

5.6 Development Process

The project was built incrementally in phases, each ending with a working application, automated tests and a Git commit, as listed in Table 5.7. Design changes from the initial plan were made in response to measured problems rather than in advance: the re-ranker after the Apache Pig failure, the switch to Claude after repeated Gemini failures, image reading after inspecting the cohort's notes, question splitting and partial answers after the browser check, and shared sessions after the planner dry run. Every phase was verified before the next began, and the final phase verified the whole system on the real AI cohort in a headless browser. The complete history of the work, with one commit per change and a message explaining it, is available in the GitHub repository given in Appendix A, which allows each design decision to be traced to the problem that motivated it.

Table 5.7 Development phases

Phase	Deliverable
1. Set-up	Virtual environment, configuration, LLM wrapper with fallback, SQLite schema
2. Notes Q&A	Extraction, chunking, embeddings, ChromaDB, cross-encoder re-ranking, cited answers, topics
3. Quiz	Topic-based MCQ generation, validation, shuffling, answer-key check, grading, saving answers
4. Memory and plan	Topic scores, weak/strong status, adaptive mode, progress page, rule-based study plan
5. Evaluation	RAG vs no-RAG on 60 examination questions with a blind judge; adaptive-quiz simulation
6. Claude and cohorts	Claude as primary provider; cohorts, hall-ticket accounts, admin page, timetable planner, quiz by chapter
7. Security	One-time codes, PBKDF2 hashes, lockout; browser checks with Playwright
8. Real cohort	PowerPoint and image-page reading, topic coverage, multi-part answers, AI cohort set-up and verification
9. Engagement and polish	Shared sessions, calendar export, email updates, email at sign-up, professional interface redesign

CHAPTER 6
RESULTS AND DISCUSSION

6.1 Evaluation Metrics

The main task, open-ended question answering, has no fixed set of classes, so classification metrics such as precision, recall and F1-score over classes do not apply directly. Instead, each answer is graded against the reference answer from the course question bank as correct, partial or wrong, and a reply saying that the notes do not contain the answer counts as wrong. For N questions with C correct answers, P partial answers and A abstentions, Equation (6.1) defines the accuracy, which is the headline metric of the evaluation; Equation (6.2) defines a partial-credit score that gives half a mark to each partial answer; and Equation (6.3) defines the accuracy among the questions the system chose to answer, which separates wrong answers from honest abstentions. Because both conditions of every question are graded in the same call by a blind judge, the difference between the two conditions is more reliable than either absolute value, which depends on how strict the judge is.

$$\text{Accuracy} = C / N \tag{6.1}$$

$$\text{Score} = (C + 0.5 \cdot P) / N \tag{6.2}$$

$$\text{Answered accuracy} = C / (N - A) \tag{6.3}$$

For the quiz simulation, the metrics are the share of questions placed on genuinely weak topics, the share of weak topics detected within ten quizzes, and the mean mastery of weak topics after ten quizzes, where answering a question on topic t together with its explanation is assumed to raise its mastery m_t as in Equation (6.4). For the end-to-end verification on the AI cohort, every check is a pass or a fail decided in advance: an answer passes only if it cites the file that was named before the run, a quiz passes only if it has ten valid and distinct questions whose sources lie in the chosen chapter, and the isolation check passes only if a student of another cohort sees none of the cohort's notes. Response times are reported for every operation, because a study assistant that takes a minute to answer is not used in practice, however accurate it is.

$$m_t \leftarrow m_t + 0.08 \cdot (1 - m_t) \tag{6.4}$$

6.2 Training / Validation Behaviour

Because no model was trained, there are no training or validation loss curves, and two analyses take their place. The first checks the one decision the pipeline makes before calling any language model: whether any passage is relevant enough to answer from. Fig. 6.1 shows the best cross-encoder score among the twenty candidates for each of the 60 evaluation questions and for 15 off-syllabus questions, such as “What is the capital of France?” and “Explain Ohm’s law.”, computed on the evaluation index that contains only the two unit PDFs. All 60 examination questions scored above the threshold of −5, with a lowest score of -1.27 and a median of 4.65, so none was refused at this stage, while 14 of the 15 off-syllabus questions scored below it (median -10.99) and were refused without any model call. The one exception, “Explain HDFS block replication.”, is a near-domain Big Data question that reaches the model, whose NOT_FOUND instruction is the second safeguard.

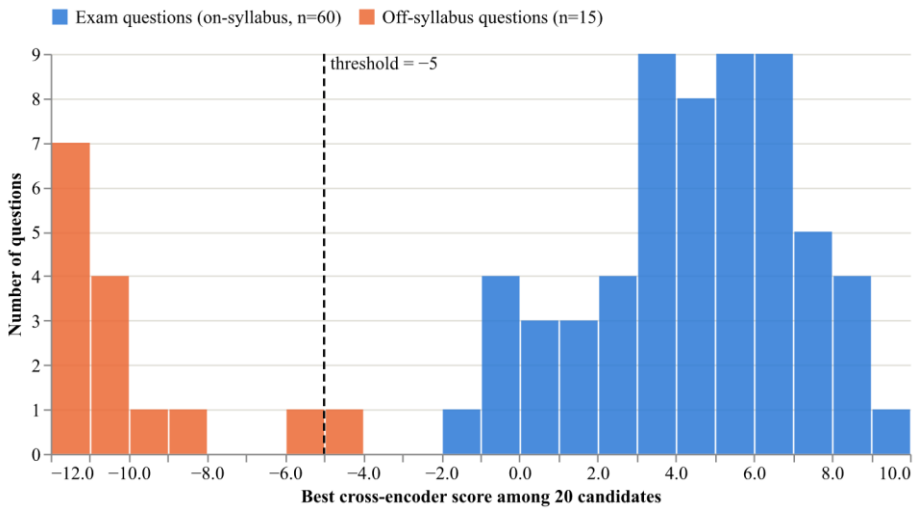


Fig. 6.1 Re-ranker calibration: best passage score for on-syllabus and off-syllabus questions

The second analysis compares the development set, which was used to find problems and to try changes, with the held-out set, which was run on the final pipeline only. If design choices had been over-fitted to the development questions, the held-out accuracy would be clearly lower than the development accuracy. The opposite was observed in the Claude run, where notes-grounded accuracy was 67 % on the development set and 77 % on the held-out set, and in the Gemini run the two were within one question of each other (87 % and 83 %). Differences of one or two questions are within the variation of the judge, which was measured by re-judging unchanged answers (Section 6.6), so there is no sign of over-fitting. The development set is slightly harder for retrieval

because it contains three definitions of Pig data types that the indexed notes list without explaining, a pattern analysed in the error analysis below.

6.3 Test-Set (Held-Out) Results

Table 6.1 gives the results of both evaluation runs on both halves of the question set; the held-out rows are the test-set result of each run, and Fig. 6.2 shows the same results graphically. The first run used Gemini 3.5 Flash-Lite, the main model of the system at the time, and the second run used Claude Sonnet 5.5, the main model of the final system, on the same questions and the same index. The two runs therefore differ in two ways at once: the model that answers and judges, and the pipeline, because the second run includes the question-splitting stage and the partial-answer rule of Sections 4.6.1 and 4.6.2, which did not exist at the time of the first run. The comparison is therefore a comparison of two complete versions of the system rather than of two models in isolation, and the error analysis in Section 6.6 separates the two effects by reading every answer that was not fully correct.

Table 6.1 Answer accuracy with and without the notes in both runs (held-out rows are the test result)

Run	Set	Condition	Correct	Partial	Wrong	Accuracy	Score
Gemini 3.5 Flash-Lite	Held-out (30)	With notes (RAG)	25	5	0	83 %	92 %
		Without notes (B1)	30	0	0	100 %	100 %
	Development (30)	With notes (RAG)	26	1	3	87 %	88 %
		Without notes (B1)	29	1	0	97 %	98 %
Claude Sonnet 5.5	Held-out (30)	With notes (RAG)	23	5	2	77 %	85 %
		Without notes (B1)	28	2	0	93 %	97 %
	Development (30)	With notes (RAG)	20	6	4	67 %	77 %
		Without notes (B1)	28	2	0	93 %	97 %

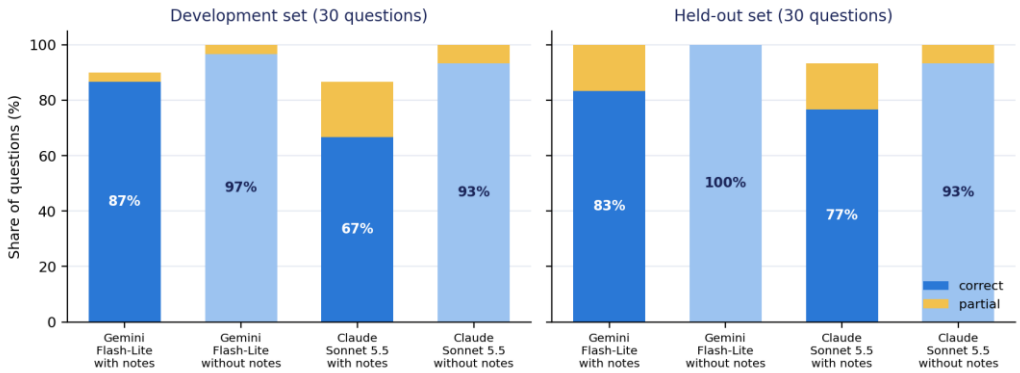


Fig. 6.2 Correct and partial answers with and without the notes, for both runs and both halves

On the held-out set, the final system answered 23 of 30 questions fully correctly from the notes, 5 partially and 2 wrongly, a partial-credit score of 85 %, while the same model without notes answered 28 correctly. In the earlier Gemini run the corresponding figures were 25 correct with notes and 30 without. In both runs, therefore, the plain model was more complete on these questions than the notes-grounded answers, and the Claude version of the system scored somewhat lower than the Gemini version on the grounded side. Table 6.2 breaks the results down by unit, and Table 6.3 compares the answers themselves: the length of the answers in words and the number of grounded answers that explicitly state that the notes do not cover part of the question, which is the behaviour introduced by the partial-answer rule.

Table 6.2 Correct answers by unit (15 questions per unit and set)

Run	Set	Unit	With notes	Without notes
Gemini Flash-Lite	Held-out	III (Pig, Flume)	14	15
		IV (Hive, Sqoop)	11	15
	Development	III (Pig, Flume)	13	14
		IV (Hive, Sqoop)	13	15
Claude Sonnet 5.5	Held-out	III (Pig, Flume)	13	14
		IV (Hive, Sqoop)	10	14
	Development	III (Pig, Flume)	9	14
		IV (Hive, Sqoop)	11	14

Table 6.3 Answer length and explicit statements of what the notes do not cover

Run	Set	Median words with notes	Median words without notes	Grounded answers naming a gap
Gemini Flash-Lite	Held-out	85	137	0
Gemini Flash-Lite	Development	80	139	3
Claude Sonnet 5.5	Held-out	151	268	8
Claude Sonnet 5.5	Development	127	253	10

Two observations follow from Table 6.3. First, notes-grounded answers were roughly half the length of the answers written without notes in both runs, because they contain only what the excerpts support, which makes them quicker to check against the cited pages. Second, the Claude version named a gap in 8 of the 30 held-out answers and 10 of the development answers, against 0 and 3 in the Gemini run, where the gaps in the development set were whole-question abstentions. When the notes list a term without defining it, the final system says so instead of quietly supplying a definition from the model's own knowledge, and the judge, comparing against a reference answer that contains the definition, grades such an answer as partial or wrong. This behaviour lowers the measured accuracy but is exactly what the grounding instructions ask for, and it is the main reason for the lower grounded score of the Claude run, as the case-by-case analysis in Section 6.6 shows.

6.4 End-to-End Verification on the AI Cohort

The accuracy evaluation measures one component on one course; the end-to-end verification measures the complete application on the real notes of the AI cohort, through a real browser, exactly as a student would use it. A script drove headless Chromium against the running application: it signed in as two demonstration students with their one-time codes, asked two questions per subject and checked that each answer cited the file named in advance, asked one question spanning two subjects and one question the notes do not cover, checked that the quiz chapter picker offered every one of the seventeen chapters, generated a quiz in the browser, and finally created a student of another cohort and checked that it saw none of the AI cohort's notes. Table 6.4 lists the checks; all 17 passed, and the two students used were afterwards reset with fresh codes.

Table 6.4 Browser checks on the AI cohort (headless Chromium, real models)

Check	Time / detail	Result
24B81A6601 signs in with their code	—	Pass
NLP: "What is a context-free grammar?" cites NLP - Unit 3.pdf	7.9s	Pass
NLP: "What is coreference resolution?" cites NLP - Unit 5.pdf	6.6s	Pass
Deep Learning: "What do striding and padding do in a convolutional neural network?" cites Deep Learning - Unit 2.pptx	7.8s	Pass
Deep Learning: "How is an LSTM different from a vanilla RNN?" cites Deep Learning - Unit 3.pptx	5.5s	Pass
Big Data: "What are the main components of YARN?" cites Big Data - Unit 2.pptx	5.5s	Pass
24B81A6602 signs in with their code	—	Pass
Big Data: "What is Apache Pig used for?" cites Big Data - Unit 3.pptx	6.6s	Pass

Check	Time / detail	Result
Cyber Security: “How can SQL injection be prevented?” cites Cyber Security - Unit 3	6.9s	Pass
Cyber Security: “What are web exploit tools and how do they hide exploits?” cites Cyber Security - Unit 4.pptx	9.4s	Pass
Negotiation Skills: “What is the difference between distributive and integrative negotiation?” cites Negotiation Skills - Unit 2.docx	6.4s	Pass
Negotiation Skills: “Why do people negotiate?” cites Negotiation Skills - Unit 1.docx	5.0s	Pass
cross-subject question cites both subjects' files	8.5s	Pass
question the notes don't cover is answered honestly	2.9s	Pass
quiz picker lists every subject's chapters	17 of 17; missing []	Pass
10-question quiz for Deep Learning · Unit 2 generated in the browser	23.8s	Pass
a student of another cohort sees none of the AI cohort's notes	—	Pass

Quiz quality was audited separately by generating a ten-question quiz for one chapter of each subject through the same code path as the Quiz page and reading every question. Table 6.5 lists the results: every quiz contained ten valid questions, none was a duplicate, and the source of every question lay inside the chosen chapter. Reading the fifty questions showed correct answer keys and plausible wrong options taken from the same topic area, and individual facts were spot-checked against the excerpts, including the complexity of the CKY parsing algorithm, which came from a page recovered by the image-reading stage and would not have existed in the index without it. Two Negotiation Skills questions referred to “the PPT” and “the supporting document”, wording copied from the notes themselves, which is a minor stylistic flaw rather than an error. A complete quiz took between 16 and 21 seconds, of which the single model call took between 12 and 16 seconds.

Table 6.5 Chapter quiz audit (10 questions each, Claude Sonnet 5.5)

Chapter	Valid questions	Duplicates	Sources in chapter	Total time	Model time
NLP · Unit 3	10	0	Yes	19.7 s	14.5 s
Deep Learning · Unit 3	10	0	Yes	16.1 s	12.1 s
Big Data · Unit 2	10	0	Yes	17.8 s	14.5 s
Cyber Security · Unit 3	10	0	Yes	20.8 s	15.5 s
Negotiation Skills · Unit 1	10	0	Yes	19.9 s	14.5 s

Fig. 6.3 shows, for every file of the AI cohort, how many pages or slides had extractable text and how many were image-only and were read by the vision model. In total 371 pages and slides were recovered, which used about 602,000 input tokens and 93,000 output tokens, or about 2.10 US dollars for the whole cohort, paid once at upload. Without this stage, NLP Units 3, 4 and 5 would have been indexed from only a few

hundred words each, and several questions in the verification, including the CKY fact in the quiz audit, could not have been answered. The only file whose image slides yielded no text was the Cyber Security Unit 3 deck, whose eighteen picture slides contained decorative illustrations without any words. Table 6.6 summarises the response times measured in the browser and in the quiz audit: single questions were answered in 5.0 to 9.4 seconds, the two-part question in 8.5 seconds, the question outside the notes in 2.9 seconds because no model was called, and ten-question quizzes in 16 to 24 seconds.

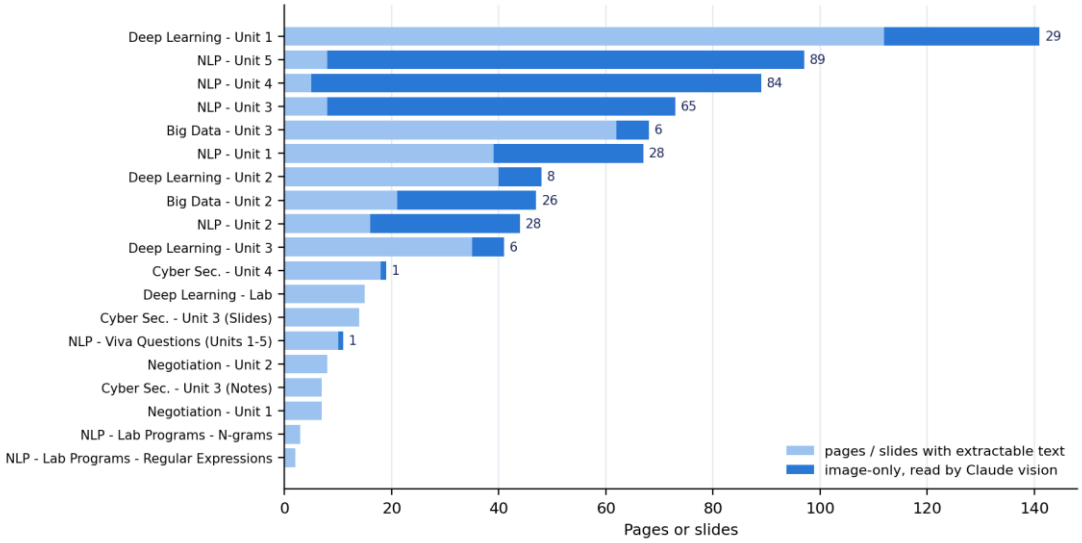


Fig. 6.3 Pages or slides per file of the AI cohort, and how many were image-only and read by the vision model

Table 6.6 Observed response and processing times on the development machine

Operation	Observed time
Answer a single question (Claude, browser)	5.0–9.4 s (10 questions)
Answer a two-part question across two subjects (browser)	8.5 s
Reply to a question the notes do not cover (no model call)	2.9 s
Generate a 10-question chapter quiz (Claude)	16.1–20.8 s (5 chapters); 23.8 s in the browser
Ingest the 19 AI-cohort files including image reading	≈ 335 s in total; 3–53 s per file
Load the embedding and re-ranking models after server start	≈ 30 s, once
Generate a 5-question quiz with the local model (offline)	≈ 97–100 s including the answer-key check

6.5 Comparison with Baselines and Published Work

6.5.1 RAG against the LLM without Notes (B1)

On these examination questions the plain model was more complete than the notes-grounded answers in both runs, which is the opposite of the naïve expectation and needs

explaining rather than hiding. The questions are general knowledge for a modern LLM: “What is Apache Hive?” or “What is a Tuple in Pig?” are well covered in its training data, so the answer without notes is complete, whereas the grounded answer can only use what the retrieved excerpts contain, and the indexed notes sometimes list a term without defining it. Where the course does have specific points, grounding helped: in the Gemini run, for “What are the benefits of using schemas?”, the grounded answer gave the course’s own points and was graded correct while the plain answer gave generic database benefits and was graded partial. No grounded answer in either run stated something that contradicted the notes; every non-correct verdict was an omission or an explicit statement that the notes did not cover the point.

This evaluation therefore does not measure the two benefits that motivated grounding in the first place: answers that follow the course’s own terminology and emphasis, and the ability of the student to verify every claim against a cited page. A question set built from material that is not general knowledge, such as a lecturer’s own examples, numerical problems or the scanned NLP units of the AI cohort, would be a fairer test of that value, and the end-to-end verification of Section 6.4 provides qualitative evidence in that direction, since every one of its answers cited the correct file of a real course. A practical consequence is that an optional, clearly labelled mode that adds general knowledge after the notes-only answer would combine the completeness of the plain model with the verifiability of grounding, and this is listed as future work in Section 7.3.

6.5.2 Adaptive against Even-Mix Quizzes (B4)

A real comparison of quiz strategies would require many students over several weeks, so the adaptive strategy was evaluated by simulation using the application’s own allocation and scoring code, with the set-up given in Table 4.1 and the learning assumption of Equation (6.4). Table 6.7 gives the results and Fig. 6.4 shows the mastery of the weak topics after each quiz. Adaptive mode spent more than twice as much practice on the weak topics once they were identified and, under the stated assumption, raised their mastery by about thirteen points more than an even mix. It did not identify weak topics much faster, because a topic is judged after only three answers, and a weak topic with about 40 per cent accuracy still scores two out of three by chance about a third of the time. The size of the mastery gain depends on the assumed learning rate,

while the share of questions on weak topics does not, so the second number is the more robust result of the simulation.

Table 6.7 Simulation results: adaptive versus even-mix quizzes (200 simulated students per mode)

Measure	Adaptive	Even mix (B4)
Questions on genuinely weak topics (quizzes 4–10)	55 %	25 %
Weak topics flagged as weak at least once in 10 quizzes	73 %	70 %
Students whose weak topics were all flagged within 10 quizzes	18 %	10 %
Quizzes needed to flag all weak topics (mean, when flagged)	6.4	8.5
Mean mastery of weak topics after 10 quizzes (start ≈ 35 %)	68 %	55 %

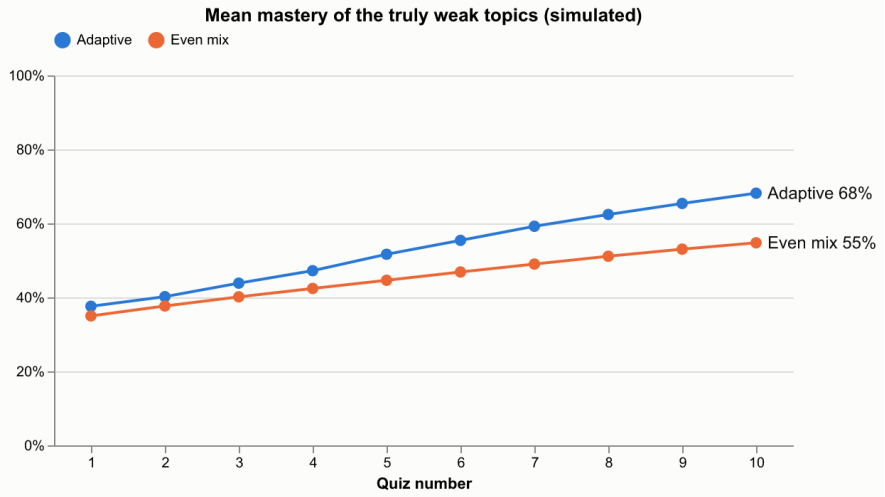


Fig. 6.4 Simulated mastery of the genuinely weak topics per quiz

6.5.3 Relation to Published Work

The numbers above cannot be compared directly with published benchmarks, because those use different data and tasks, but the findings are consistent with the literature. The advantage of RAG reported by Lewis et al. [3] is largest for knowledge the model does not already have; on general-knowledge course questions the plain model is a strong baseline. The error analysis below agrees with the RAG survey [8] that chunking and retrieval quality, rather than generation, dominate end-to-end accuracy. The large effect of the re-ranker on the Apache Pig question mirrors the gains reported by Nogueira and Cho [14], the use of a few well-ranked passages and the interleaved merge follow Liu et al. [18], and the cross-subject failure fixed by question splitting is a retrieval-side instance of the compositionality gap described by Press et al. [21], where a question that combines two facts fails even though each fact on its own is answered correctly.

6.6 Error Analysis

Table 6.8 lists every notes-grounded answer of the Claude run that was not fully correct, together with its cause, found by reading the answer, the judge's reason and the retrieved excerpts. Three groups stand out. The largest is definitions that the notes list but do not explain, such as the Pig data types Tuple, Map and Bag, where the system correctly said that the notes did not define them and was graded wrong. The second is retrieval that found a related but not the defining passage, as with Hive's conditional functions and data import, where the word "import" retrieved the Sqoop pages instead of Hive's LOAD DATA section. The third is answers that covered the main point but omitted items of a list or the example in the reference, which the stricter Claude judge graded as partial. None of the non-correct answers contradicted the notes, so the errors are errors of omission, and the student is told about most of them in the answer itself.

Table 6.8 Notes-grounded answers of the Claude run that were not fully correct

Question (set)	Verdict	Cause
What is a Tuple / Map in Pig? (dev); What is a Bag in Pig? (held-out)	Wrong	The notes name the data types but do not define them; the answer said so instead of adding outside knowledge
What are conditional functions in Hive? (dev)	Wrong	The IF/CASE passage was not among the retrieved candidates; answer stated the gap
What is a Subquery in Hive? (dev)	Wrong	Only a passage that mentions subqueries passed the threshold; replied "not found"
How can data be imported into Hive? (held-out)	Wrong	"Import" retrieved the Sqoop pages instead of Hive's LOAD DATA and INSERT sections
GROUP/COGROUP, UNION/SPLIT, JOIN (dev, held-out)	Partial	Main idea correct; the key-based grouping, routing conditions or example missing from the excerpts
Relational operators, UDFs, mathematical functions in Hive (held-out)	Partial	Retrieved excerpts listed fewer items than the reference (no UDAF/UDTF; no LIKE/RLIKE/IN)
Bucketing, Schema-on-Read, Managed vs External tables (dev, held-out)	Partial	Chunk boundary cut the CLUSTERED BY example or the third difference
FILTER operator, Filter UDF (dev)	Partial	Purpose correct; the judge expected the boolean-return detail

The common root cause of the retrieval failures is chunks that lose their context: a 180-word chunk from the middle of a section does not contain the section heading, such as "Pig Latin – Data Types" or "Hive – Importing Data", that would connect it to the question, and Unit IV is harder because Hive and Sqoop share vocabulary such as "import" and "table". The same cause explains the one weakness seen on the AI cohort, where the answer to a question about striding and padding explained padding but said that the notes did not explain striding, although slide 10 of the Deep Learning Unit 2 deck defines stride; that slide's re-ranker score was about −5.9, just below the threshold,

because the slide uses the word “stride” in a formula rather than in a sentence. Prefixing every chunk with its section or slide heading and adding keyword matching for exact terms are therefore the most promising fixes, and both are listed as future work.

The reliability of the judge was also examined. In the keyword-search experiment of the first run, the unchanged answers written without notes were judged again and scored 27 correct instead of 29, so the judge varies by about two questions in thirty, and differences of one or two questions between runs are not meaningful. The two judges also differ in strictness: the Claude judge graded more omissions as partial, which lowered both conditions in the second run, including the answers written without notes. The gap between the two conditions within each run is larger than this variation, so its direction is reliable for this question set. Because the judge is the same model that wrote the answers, a self-preference bias is possible [19]; grading against a written reference, blind ordering and the columns provided for manual overrides in the results files limit it, and a full manual check of the verdicts would strengthen the result further.

6.7 Ablation Study

Table 6.9 summarises the effect of removing or changing individual components. Only the first two rows are controlled measurements on the full development set; the others were observed on specific cases during development or verification, or follow from the specification of a component, and they are reported as such rather than presented as averages. Every row records the decision that was taken as a result, so the table also documents why the final system looks the way it does. Taken together, the ablations show that the components that most changed the behaviour of the system on real notes were the re-ranker, the image-reading stage, the question splitting and the partial-answer rule, while the alternatives that were expected to help, keyword search and a larger re-ranker, made little or no difference on the development questions and were therefore not adopted, keeping the system simpler.

Table 6.9 Ablation study

Variant	Evidence	Result	Decision
Add BM25 keyword search to stage 1 (B2)	Development set, end to end (Gemini run)	RAG 25 vs 26 correct (no gain)	Reverted
Larger re-ranker MiniLM-L-12 (B3)	Development set, retrieval only	28/30 vs 27/30 questions retrieved	Not adopted
Remove the cross-encoder	“What is Apache Pig?”	Defining chunk ranked 8th → not found; with re-ranker 1st	Kept

Variant	Evidence	Result	Decision
Remove image reading (B7)	AI cohort ingestion	371 pages and slides empty; NLP Units 3–5 nearly unsearchable	Kept
Single search for two-part questions (B6)	Browser check, YARN + pooling	Only Big Data cited; pooling reported missing	Split and merge
Remove the partial-answer rule	Browser check, striding + padding	Whole answer refused although padding was found	Rule kept
Keep topics listed only in a syllabus	Review of extracted topics	Quizzes on topics the notes never explain	Coverage filter
One session per topic always	Planner dry run, AI cohort	Every realistic timetable refused	Shared sessions
All-time accuracy instead of last 10 (B5)	Worked example / unit test	10 wrong then 9 right: 47 % (weak) vs 90 % (strong)	Last 10 answers
500-token chunks instead of 180 words	Model specification	Text beyond 256 word-pieces silently ignored	180 / 40 words

6.8 Output Screenshots

The screenshots in this section were taken automatically from the running application by a browser script on the day this report was built, using the real AI cohort and a temporary demonstration student who was deleted afterwards, so they show the system exactly as it behaves. They follow a complete session: the administrator reviews the subjects and notes shared with the cohort; a student signs in for the first time with a one-time code and an email address, asks questions that are answered from the notes with citations, including a question across two subjects and one the notes do not cover, takes a chapter quiz and reviews the result, sees the accuracy of every topic on the Progress page, and creates a study plan for the examination timetable, and the last figure shows the same application on a phone. In the quiz shown, the script deliberately chose mostly the first option, so the low score illustrates how wrongly answered topics are tracked rather than how well a student performs; after a single quiz each topic has only one answer, so the Progress page lists these topics as still being judged until three answers have been given.

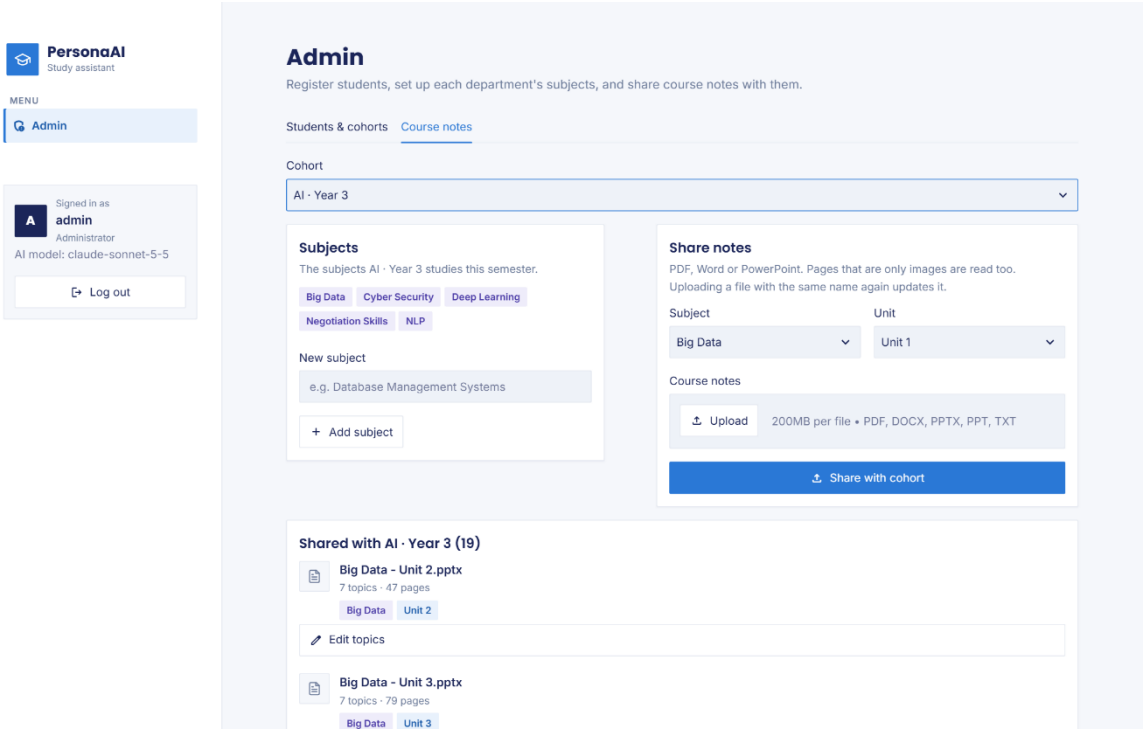


Fig. 6.5 Admin: subjects and the course notes shared with the AI cohort, by unit

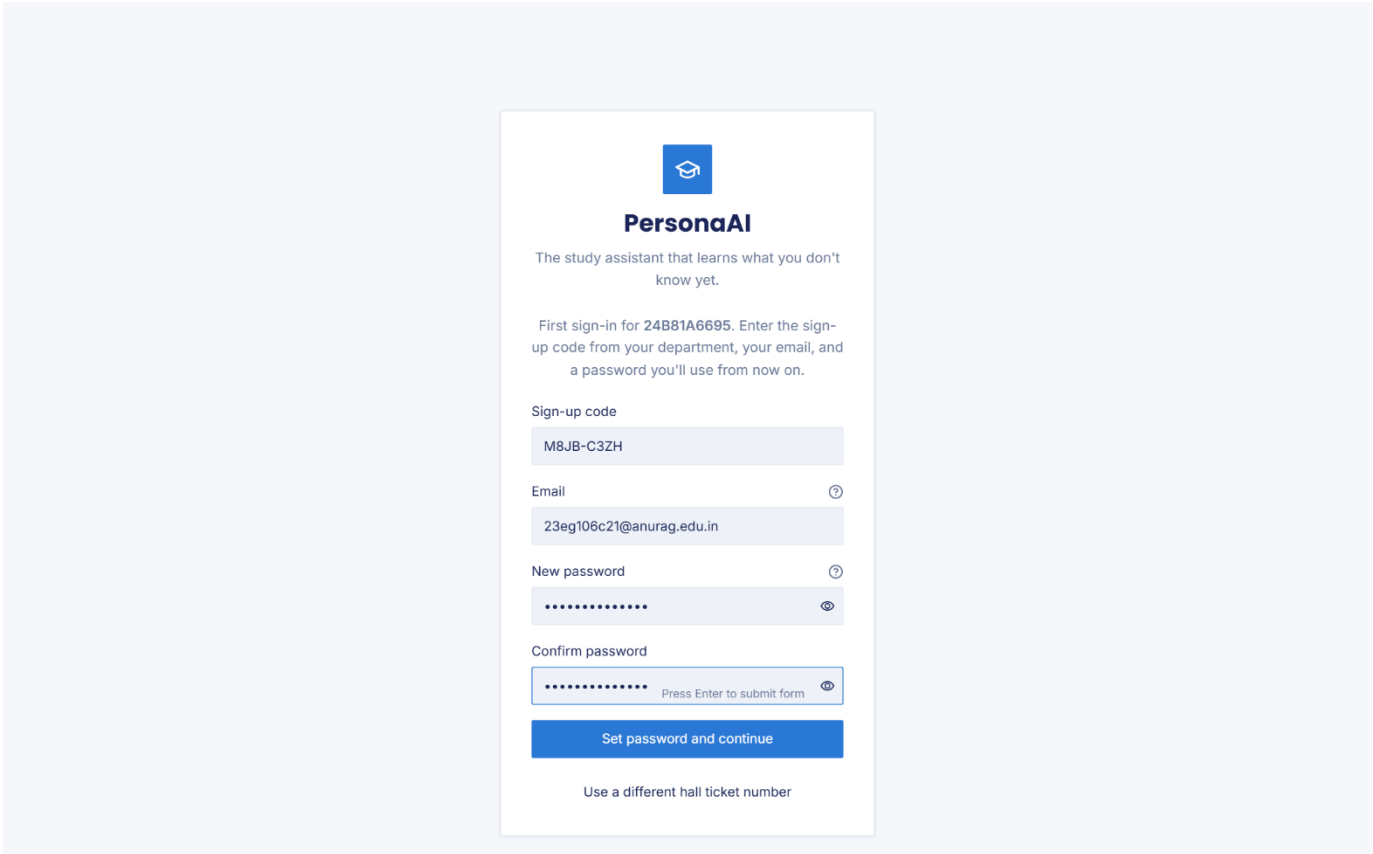


Fig. 6.6 First sign-in: one-time sign-up code, email address and a new password

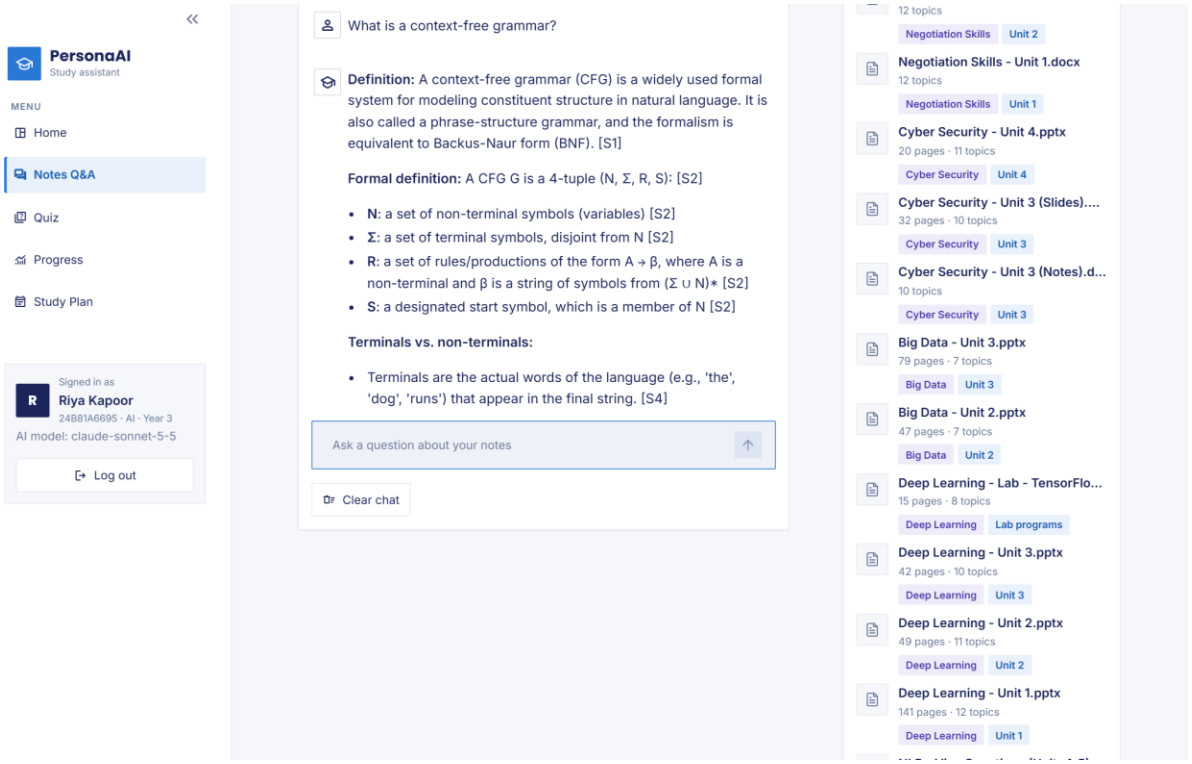


Fig. 6.7 An answer grounded in the notes, with inline citations

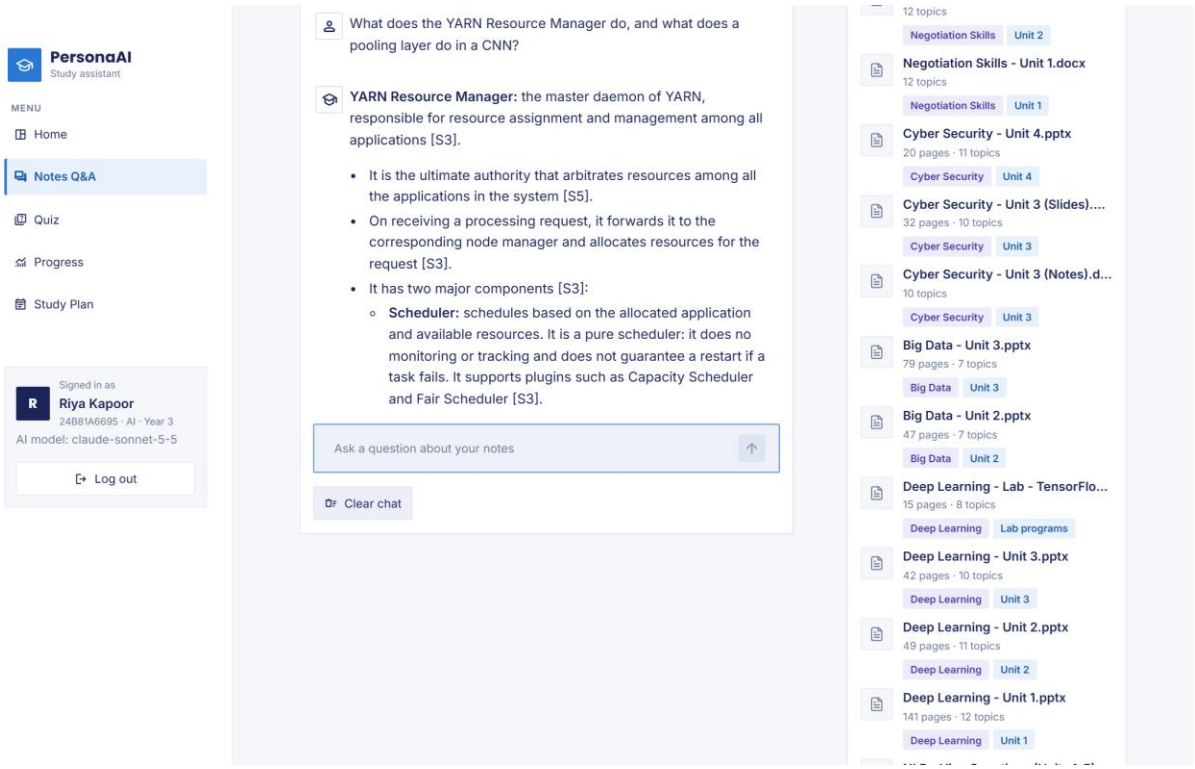


Fig. 6.8 A two-part question answered from two subjects' notes

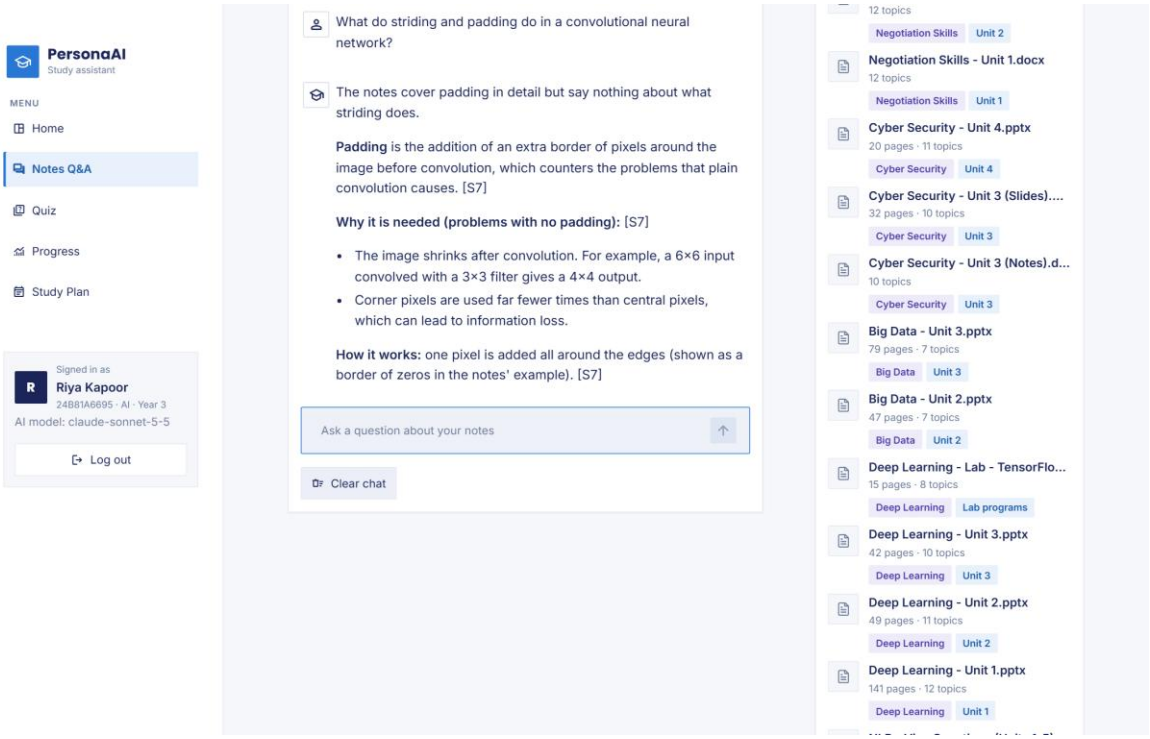


Fig. 6.9 An answer citing PowerPoint slides that names the part the notes do not explain

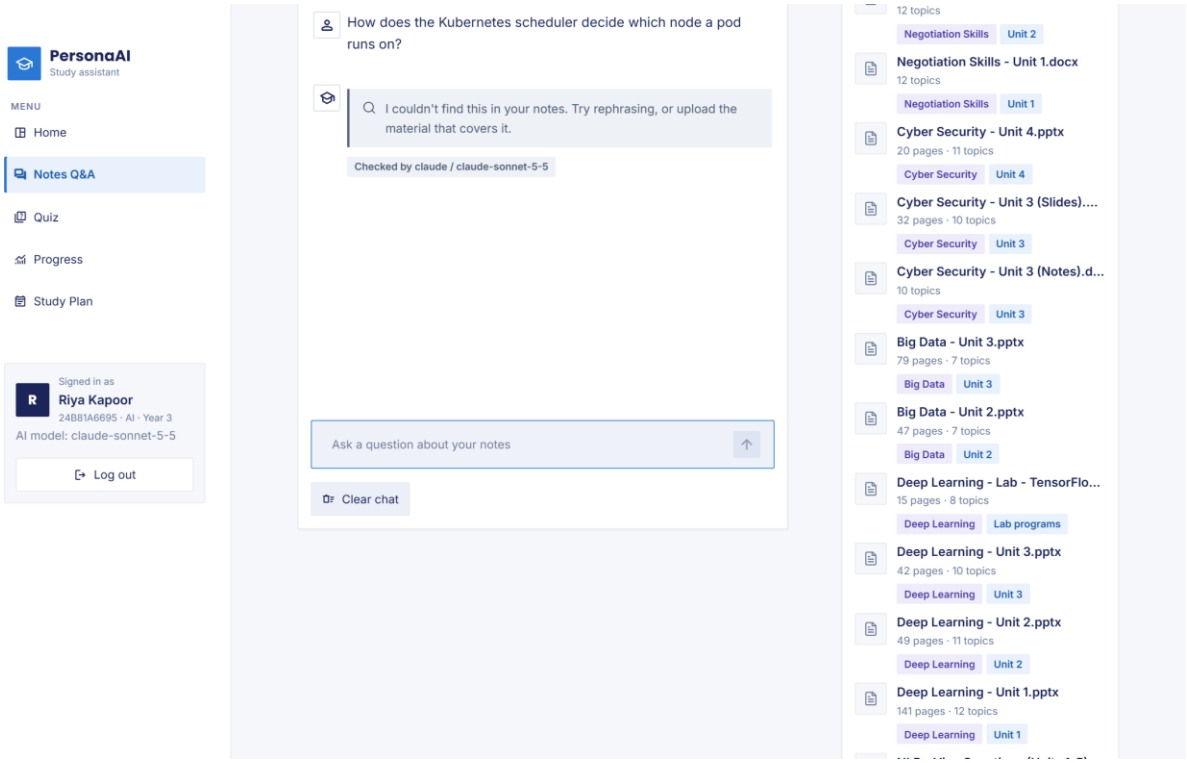


Fig. 6.10 A question the notes do not cover is declined instead of answered from general knowledge

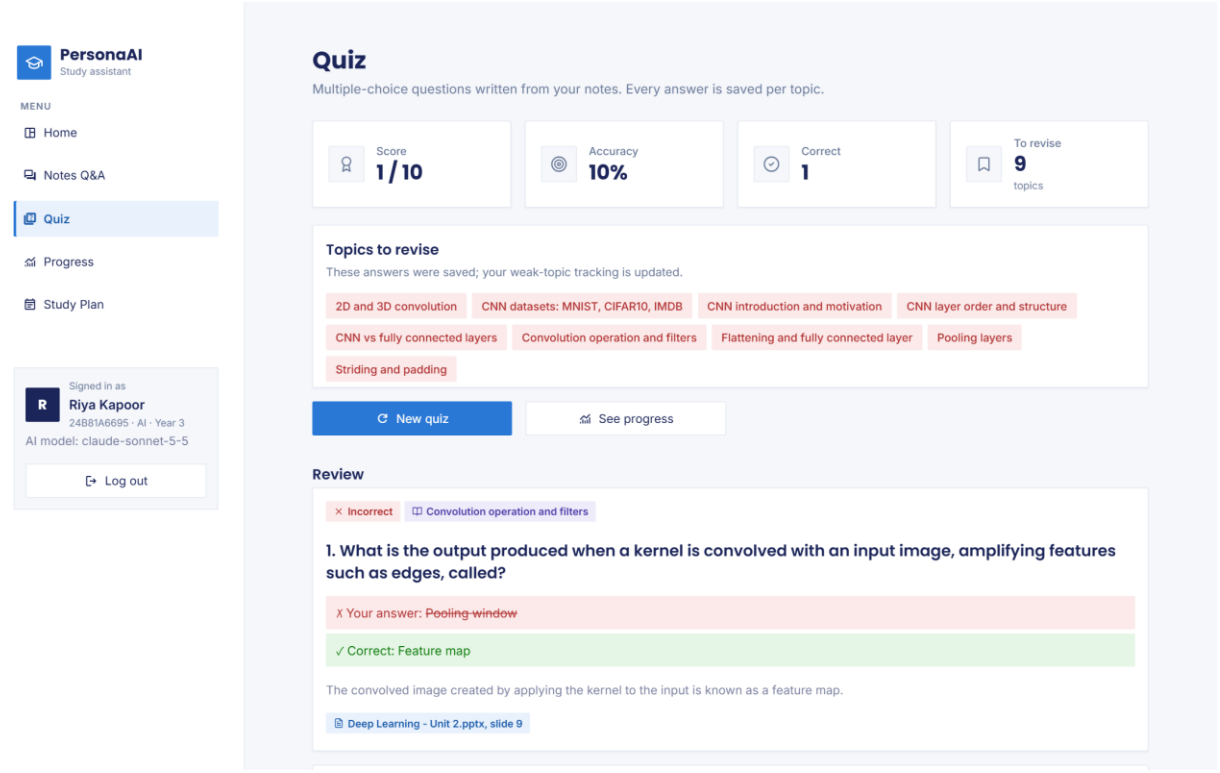


Fig. 6.11 Quiz results: score, topics to revise and the review of each question with its source

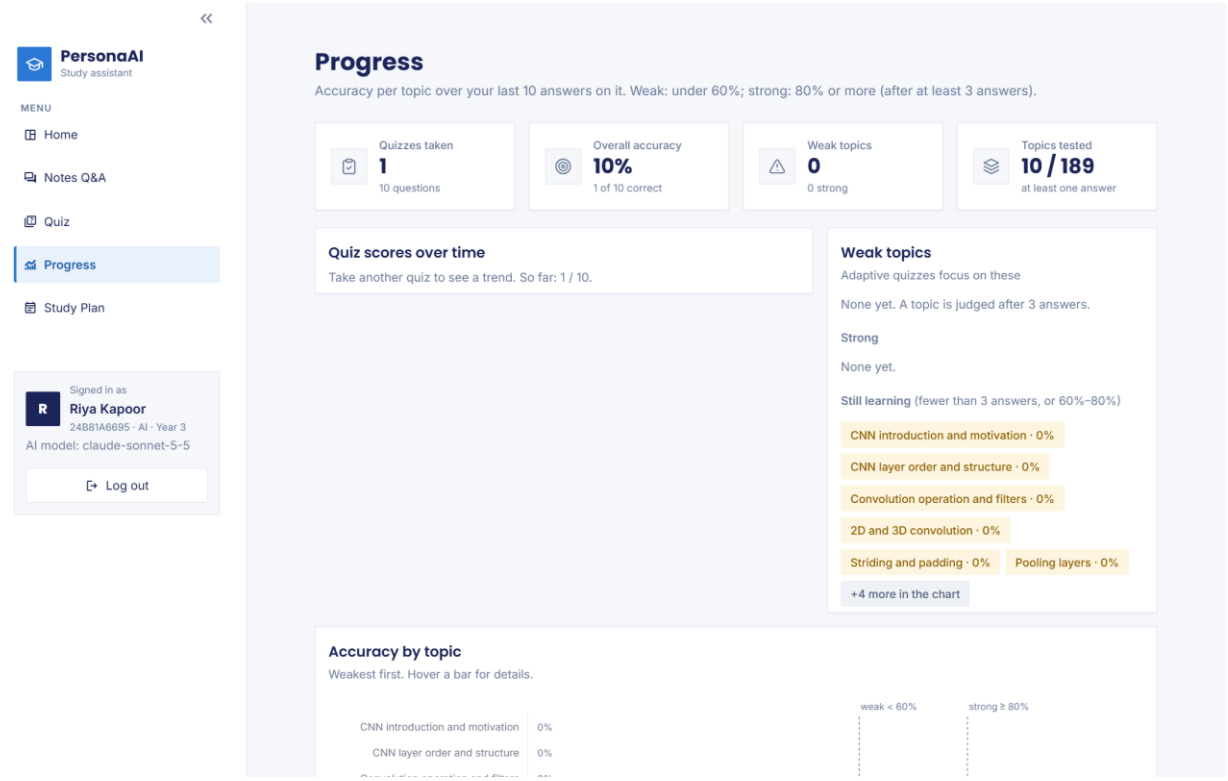


Fig. 6.12 Progress page: score trend and topic accuracy, with topics still being judged after one quiz

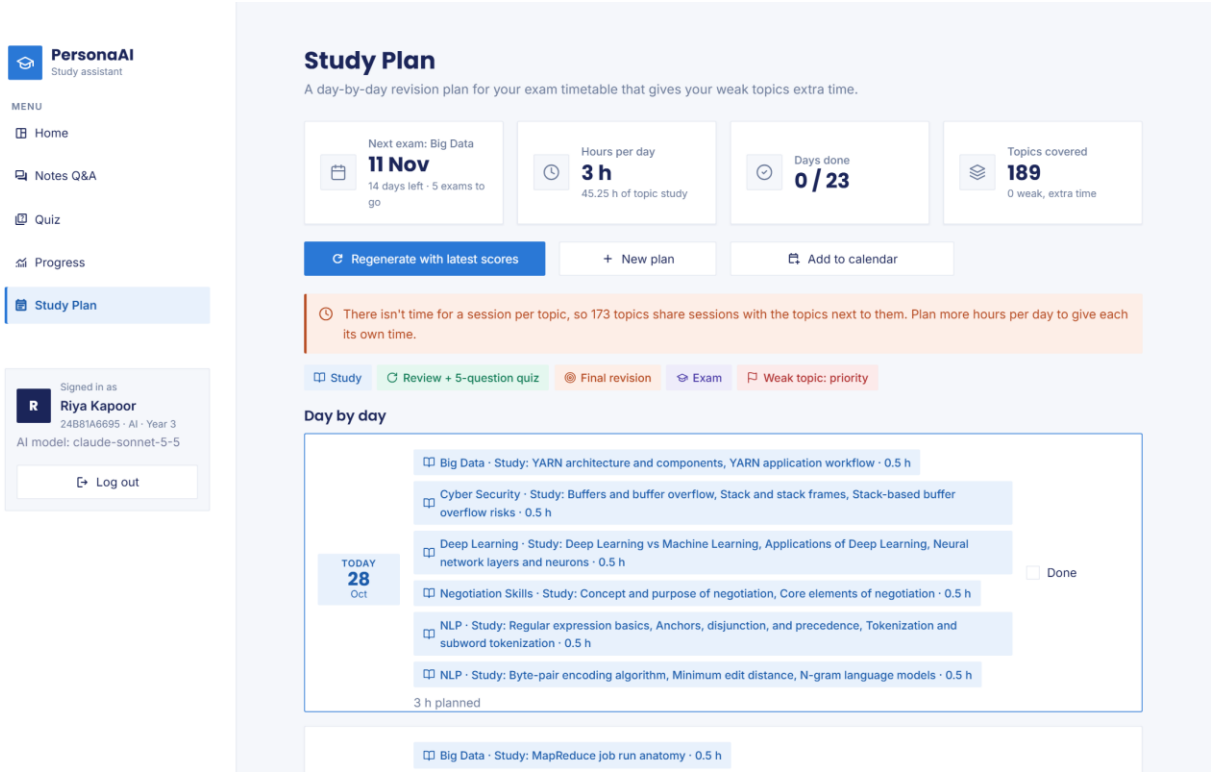


Fig. 6.13 Study plan: next examination, statistics, shared-session note and Add to calendar

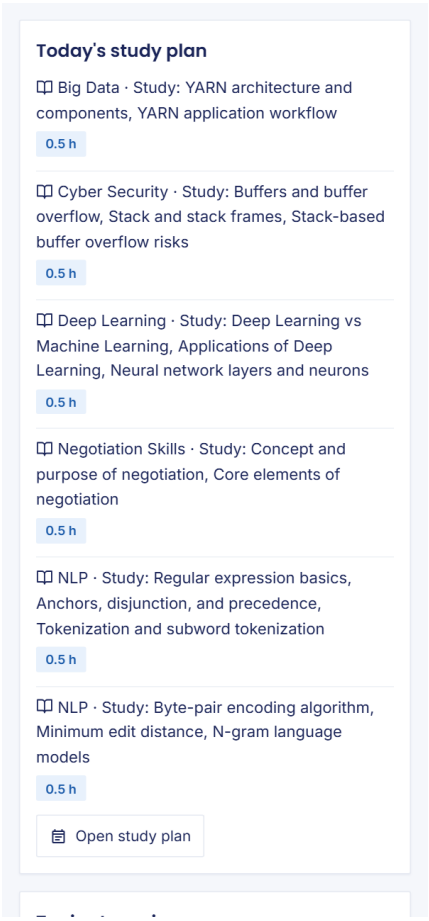


Fig. 6.14 Home page at phone width

6.9 Ethical and Responsible AI

Data privacy was treated as a design requirement. Uploaded notes, the database and the vector index are stored on the department's own machine and are excluded from the repository, and every query is filtered by the student's user and cohort, which was verified in the browser with a student of another cohort. In online mode, the retrieved excerpts, the question and, for image-only pages, the page images are sent to the Claude API, a third-party service, so the department should share only course material and students should not upload personal or confidential documents; the offline mode keeps all processing on the computer. Passwords are stored only as salted hashes, sign-up codes are shown once and stored as hashes, email addresses are used only for the updates the student chooses, every email explains how to stop them, and API keys, the administrator password and the email password are kept in a git-ignored file and never written to the log.

Hallucination and over-reliance remain risks even with grounding [2]. Inline citations, the source excerpts under every answer and the explicit statement of what the notes do not cover encourage students to check answers rather than trust them blindly, in line with the concerns raised by Kasneci et al. [1]. The evaluation covers one course in English and sixty questions, so its results may not transfer to other subjects or languages, the judge may share biases with the answering model [19], and the learner model applies fixed thresholds to every student, which may be harsh for inherently harder topics. The benefit of adaptive quizzes is a simulation result under an assumed learning rate rather than a measured improvement in real students' marks, and it is reported as such throughout this report, so that no reader mistakes it for an experiment with students.

The tool could be misused to generate quiz questions from copyrighted material that the user has no right to share, or during closed-book assessments. It is intended for revision of course material that the department legitimately distributes to its own students, and it does not publish or share uploaded content outside the cohort. The demonstration cohort uses invented names and hall-ticket numbers, so no real student's data appears in this report, the repository or the screenshots. Status is never conveyed by colour alone and the interface works on phones, which supports accessibility, and the small local models used for embedding and re-ranking keep the energy cost of each query low, while the larger hosted model is called once per answer. The image-reading

cost is paid once per file at upload rather than for every student, which keeps the total cost of the system proportional to the amount of course material rather than to the number of students.

CHAPTER 7
CONCLUSION AND FUTURE SCOPE

7.1 Conclusion

PersonaAI shows that a study assistant can be grounded in a class's own course notes, including notes that are largely scanned or made of picture slides, can be run by a department for a whole cohort with proper accounts, can remember each student's weak topics, and can use that memory to direct both practice and the planning of a full examination timetable. Table 7.1 states, for every objective of Section 1.4, the methodology used, where it is implemented and the result that was measured. The table shows that every objective was met with evidence from automated tests, from the evaluation on the Big Data question set or from the end-to-end verification on the AI cohort, and it also records where the evidence is weaker than it could be, such as the simulated rather than real comparison of quiz strategies, so that the conclusion does not claim more than the measurements support.

Table 7.1 Objective vs Methodology vs Implementation vs Result

Obj.	Methodology	Implementation	Result
O1 Ingest notes	Page/slide extraction, Claude vision for image pages, cleaning, 180/40 chunks, topic coverage	core/documents.py, ocr.py, ingest.py, vectorstore.py	19 files, 801 pages, 371 image pages read (≈ $2.10), 893 chunks, 189 topics; no upload errors
O2 Grounded answers	Split, retrieve 20, re-rank, top 5–8, grounded prompt, partial answers	core/rag.py, llm.py	10/10 browser answers cite the expected file; held-out 23/30 correct, 0 contradictions
O3 Valid quizzes	Chapter picker, structured output, validation, regeneration, shuffling	core/quiz.py	5 chapter quizzes: 50/50 valid, 0 duplicates, sources in chapter; 16–21 s
O4 Weak-topic memory	Last-10 accuracy; weak < 60 %, strong ≥ 80 %	core/memory.py, db.py	Unit tests pass; per-topic accuracy shown after every quiz, status after 3 answers
O5 Adaptive quizzes	≈ 60 % weak share, then untested and stale topics	memory.adaptive_allocation	Simulation: 55 % vs 25 % of questions on weak topics; mastery 68 % vs 55 %
O6 Timetable plan	Weighted day shares, revision days, shared sessions, .ics, email	core/planner.py, ics.py, notify.py	Plans for every tested timetable; calendar and email verified
O7 Cohorts and accounts	Class list, cohorts, shared notes, PBKDF2, codes, lockout	core/auth.py, db.py, views/admin.py	14 cohort/auth tests; isolation verified in the browser
O8 Evaluation	Dev/held-out, B1, blind judge, two runs; browser check	evaluation/, tests/e2e_demo_ai.py	Held-out: Gemini 83 %, Claude 77 % (with notes); 17/17 browser checks
O9 Interface and availability	Streamlit, line icons, provider chain with timeouts	app.py, ui/, views/, core/llm.py	Seven pages on desktop and phone; failures logged; 159 tests pass

In summary, all nine objectives were met. The evaluation was reported honestly: on general-knowledge examination questions, notes-grounded answers were less complete than the same model answering from its own knowledge, but they never contradicted the notes, every claim could be traced to a page or slide, and the final system says plainly what the notes do not cover instead of filling the gap from outside the course. Retrieval of the right passage was identified as the main cause of the remaining gap. On the real notes of a cohort, the image-reading stage turned hundreds of empty pages into searchable text, every browser check passed, every audited quiz was valid, and the planner produced usable plans for realistic timetables. The system is therefore ready for a trial with a real class, which is the most important next step.

7.2 Limitations

Retrieval still misses or truncates some answers because fixed-size chunks lose their section or slide headings, as the Hive import question and the stride slide show, and the threshold that protects against unrelated questions can occasionally reject a relevant passage that uses a term only in a formula. The accuracy evaluation uses a single course, sixty questions and an LLM judge without a completed manual check of the verdicts, and the second run changed the model and the pipeline at the same time, so the two effects cannot be fully separated. The benefit of adaptive quizzes was measured by simulation rather than with real students, and weak-topic detection after three answers is noisy. Pictures that contain no words cannot be turned into searchable content, as the Cyber Security Unit 3 slides showed, and the demonstration cohort's hall-ticket numbers were invented for the demonstration.

Operationally, the system depends on a paid API for its best quality; the free Gemini tier is rate-limited and the local model is slower and writes weaker quiz questions, so a department without a budget will see longer waits. Morning emails require a task scheduler on the machine that runs the application, and the calendar export is a file that must be imported again when the plan is regenerated. Sign-up codes are distributed by the administrator by hand, and email addresses given at the first sign-in are not yet confirmed with a verification code, so a student could enter someone else's address, although the recipient can switch the emails off. Finally, the application has been run on a single machine for one department and has not been load-tested with hundreds of simultaneous users, which would be necessary before a university-wide deployment.

7.3 Future Enhancements

The most valuable improvement suggested by the error analysis is to prefix every chunk with its section or slide heading and to split documents on section boundaries rather than at fixed word counts, combined with keyword matching for exact technical terms such as “stride” or “CLUSTERED BY”, and then to re-run the same development and held-out evaluation to measure the gain. Stronger embedding models and re-rankers, listed in Section 4.8, can be evaluated on the same questions at the same time. An optional, clearly labelled mode that first answers from the notes and then adds general knowledge would combine verifiability with completeness, and a manual check of every verdict, or a second human-graded question set from a course whose material is not general knowledge, would make the comparison between grounded and ungrounded answers more conclusive.

On the learning side, the threshold learner model could be replaced by Bayesian Knowledge Tracing or half-life regression once enough real answers are available, and a study with a real class over a semester would measure the effect of adaptive quizzes and plans on actual marks rather than in simulation. For the department, a faculty view of class-wide weak topics would show which units need another lecture, email verification and institutional sign-in would remove the manual distribution of codes, timetables could be imported from the university portal, and a calendar subscription link would keep phone calendars up to date without importing a file. Hosting the application on a university server with a managed database, and testing it with many simultaneous users, would complete the path from a mini project to a service that every department of the university could use.

REFERENCES

[1] E. Kasneci et al., “ChatGPT for good? On opportunities and challenges of large language models for education,” *Learning and Individual Differences*, vol. 103, art. no. 102274, 2023.

[2] Z. Ji, N. Lee, R. Frieske, T. Yu, D. Su, Y. Xu, E. Ishii, Y. J. Bang, A. Madotto, and P. Fung, “Survey of hallucination in natural language generation,” *ACM Computing Surveys*, vol. 55, no. 12, art. no. 248, pp. 1–38, 2023.

[3] P. Lewis, E. Perez, A. Piktus, F. Petroni, V. Karpukhin, N. Goyal, H. Küttler, M. Lewis, W. Yih, T. Rocktäschel, S. Riedel, and D. Kiela, “Retrieval-augmented generation for knowledge-intensive NLP tasks,” in *Proc. Advances in Neural Information Processing Systems (NeurIPS)*, vol. 33, 2020, pp. 9459–9474.

[4] H. L. Roediger and J. D. Karpicke, “Test-enhanced learning: Taking memory tests improves long-term retention,” *Psychological Science*, vol. 17, no. 3, pp. 249–255, 2006.

[5] N. J. Cepeda, H. Pashler, E. Vul, J. T. Wixted, and D. Rohrer, “Distributed practice in verbal recall tasks: A review and quantitative synthesis,” *Psychological Bulletin*, vol. 132, no. 3, pp. 354–380, 2006.

[6] J. Dunlosky, K. A. Rawson, E. J. Marsh, M. J. Nathan, and D. T. Willingham, “Improving students’ learning with effective learning techniques: Promising directions from cognitive and educational psychology,” *Psychological Science in the Public Interest*, vol. 14, no. 1, pp. 4–58, 2013.

[7] United Nations, “Goal 4: Ensure inclusive and equitable quality education and promote lifelong learning opportunities for all,” Sustainable Development Goals. [Online]. Available: https://sdgs.un.org/goals/goal4 (accessed Oct. 28, 2026).

[8] Y. Gao et al., “Retrieval-augmented generation for large language models: A survey,” arXiv:2312.10997, 2023.

[9] A. Asai, Z. Wu, Y. Wang, A. Sil, and H. Hajishirzi, “Self-RAG: Learning to retrieve, generate, and critique through self-reflection,” in *Proc. Int. Conf. Learning Representations (ICLR)*, 2024.

[10] V. Karpukhin, B. Oğuz, S. Min, P. Lewis, L. Wu, S. Edunov, D. Chen, and W. Yih, “Dense passage retrieval for open-domain question answering,” in *Proc. Conf. Empirical Methods in Natural Language Processing (EMNLP)*, 2020, pp. 6769–6781.

[11] S. Robertson and H. Zaragoza, “The probabilistic relevance framework: BM25 and beyond,” *Foundations and Trends in Information Retrieval*, vol. 3, no. 4, pp. 333–389, 2009.

[12] N. Reimers and I. Gurevych, “Sentence-BERT: Sentence embeddings using Siamese BERT-networks,” in *Proc. EMNLP-IJCNLP*, Hong Kong, China, 2019, pp. 3982–3992.

[13] W. Wang, F. Wei, L. Dong, H. Bao, N. Yang, and M. Zhou, “MiniLM: Deep self-attention distillation for task-agnostic compression of pre-trained transformers,” in *Proc. Advances in Neural Information Processing Systems (NeurIPS)*, vol. 33, 2020, pp. 5776–5788.

[14] R. Nogueira and K. Cho, “Passage re-ranking with BERT,” arXiv:1901.04085, 2019.

[15] P. Bajaj et al., “MS MARCO: A human generated machine reading comprehension dataset,” arXiv:1611.09268, 2016.

[16] L. Wang et al., “Text embeddings by weakly-supervised contrastive pre-training,” arXiv:2212.03533, 2022.

[17] S. Xiao et al., “C-Pack: Packed resources for general Chinese embeddings,” in *Proc. 47th Int. ACM SIGIR Conf. Research and Development in Information Retrieval*, 2024.

[18] N. F. Liu, K. Lin, J. Hewitt, A. Paranjape, M. Bevilacqua, F. Petroni, and P. Liang, “Lost in the middle: How language models use long contexts,” *Trans. Assoc. Computational Linguistics*, vol. 12, pp. 157–173, 2024, doi: 10.1162/tacl_a_00638.

[19] L. Zheng et al., “Judging LLM-as-a-judge with MT-Bench and Chatbot Arena,” in *Proc. Advances in Neural Information Processing Systems (NeurIPS), Datasets and Benchmarks Track*, 2023.

[20] S. Es, J. James, L. Espinosa-Anke, and S. Schockaert, “RAGAs: Automated evaluation of retrieval augmented generation,” in *Proc. 18th Conf. European Chapter of the Assoc. Computational Linguistics: System Demonstrations*, St. Julian’s, Malta, 2024, pp. 150–158.

[21] O. Press, M. Zhang, S. Min, L. Schmidt, N. A. Smith, and M. Lewis, “Measuring and narrowing the compositionality gap in language models,” in *Findings of the Assoc. Computational Linguistics: EMNLP 2023*, Singapore, 2023, pp. 5687–5711.

[22] R. Smith, “An overview of the Tesseract OCR engine,” in *Proc. 9th Int. Conf. Document Analysis and Recognition (ICDAR)*, Curitiba, Brazil, 2007, pp. 629–633.

[23] Gemini Team, Google, “Gemini: A family of highly capable multimodal models,” arXiv:2312.11805, 2023.

[24] Anthropic, “Claude API documentation: Messages, vision and structured outputs,” 2026. [Online]. Available: https://docs.claude.com (accessed Oct. 28, 2026).

[25] G. Kurdi, J. Leo, B. Parsia, U. Sattler, and S. Al-Emari, “A systematic review of automatic question generation for educational purposes,” *Int. J. Artificial Intelligence in Education*, vol. 30, no. 1, pp. 121–204, 2020, doi: 10.1007/s40593-019-00186-y.

[26] A. T. Corbett and J. R. Anderson, “Knowledge tracing: Modeling the acquisition of procedural knowledge,” *User Modeling and User-Adapted Interaction*, vol. 4, no. 4, pp. 253–278, 1994.

[27] C. Piech, J. Bassen, J. Huang, S. Ganguli, M. Sahami, L. J. Guibas, and J. Sohl-Dickstein, “Deep knowledge tracing,” in *Proc. Advances in Neural Information Processing Systems (NeurIPS)*, vol. 28, 2015, pp. 505–513.

[28] B. Settles and B. Meeder, “A trainable spaced repetition model for language learning,” in *Proc. 54th Annu. Meeting of the Assoc. Computational Linguistics (ACL)*, Berlin, Germany, 2016, pp. 1848–1858, doi: 10.18653/v1/P16-1174.

[29] D. Wu et al., “LongMemEval: Benchmarking chat assistants on long-term interactive memory,” in *Proc. Int. Conf. Learning Representations (ICLR)*, 2025.

[30] Snowflake Inc., “Streamlit documentation,” 2026. [Online]. Available: https://docs.streamlit.io (accessed Oct. 28, 2026).

[31] Chroma, “Chroma documentation,” 2026. [Online]. Available: https://docs.trychroma.com (accessed Oct. 28, 2026).

[32] N. Reimers et al., “Sentence Transformers documentation,” 2026. [Online]. Available: https://www.sbert.net (accessed Oct. 28, 2026).

[33] Google, “Gemini API documentation,” 2026. [Online]. Available: https://ai.google.dev/gemini-api/docs (accessed Oct. 28, 2026).

[34] Ollama, “Ollama: Get up and running with large language models,” 2026. [Online]. Available: https://ollama.com (accessed Oct. 28, 2026).

[35] Llama Team, AI @ Meta, “The Llama 3 herd of models,” arXiv:2407.21783, 2024.

[36] B. Desruisseaux, Ed., “Internet calendaring and scheduling core object specification (iCalendar),” RFC 5545, Internet Engineering Task Force, Sep. 2009.

[37] K. Moriarty, B. Kaliski, and A. Rusch, “PKCS #5: Password-based cryptography specification version 2.1,” RFC 8018, Internet Engineering Task Force, Jan. 2017.

[38] OWASP Foundation, “Authentication cheat sheet,” OWASP Cheat Sheet Series, 2026. [Online]. Available: https://cheatsheetseries.owasp.org/cheatsheets/Authentication_Cheat_Sheet.html (accessed Oct. 28, 2026).

[39] Microsoft, “Playwright for Python documentation,” 2026. [Online]. Available: https://playwright.dev/python (accessed Oct. 28, 2026).

APPENDICES

Appendix A: GitHub Repository Link and README

The complete source code, the tests, the evaluation scripts and their results are published at https://github.com/shiva-1301/PersonaAI. The README file describes the problem, the features, the technology stack and the architecture, lists every setting of the .env file, explains how the department administrator imports a class list, shares notes and issues sign-up codes, and gives the commands in Listing A.1 to set up the project and reproduce every result in Chapter 6. The course notes of the AI cohort and the Big Data course are not included in the repository because they are internal course material, and the .env file with the API keys, the administrator password and the email account is excluded by the .gitignore file; the evaluation results, the screenshots used in this report and the scripts that produced them are included, so that every number in Chapter 6 can be traced to a file in the repository.

Listing A.1 Set-up and reproduction commands

```
git clone https://github.com/shiva-1301/PersonaAI.git
cd PersonaAI
py -3.12 -m venv .venv
.venv\Scripts\activate          # Linux/macOS: source
.venv/bin/activate
pip install -r requirements-dev.txt
# .env: ANTHROPIC_API_KEY=..., GEMINI_API_KEY=...,
ADMIN_PASSWORD=..., SMTP_USER=..., SMTP_PASSWORD=...
streamlit run app.py
pytest                                    # 159 tests
python scripts/setup_demo_ai_cohort.py     # demo cohort (notes/
folder needed)
python tests/e2e_demo_ai.py http://localhost:8501   # browser check
python -m evaluation.run_eval --model claude-sonnet-5-5 --out
evaluation/results/dev_claude
python -m evaluation.run_eval --model claude-sonnet-5-5 \
       --questions evaluation/questions_holdout.csv --out
evaluation/results/holdout_claude
python -m evaluation.simulate_adaptive --students 200
python scripts/send_reminders.py           # morning emails
(schedule daily)
```

Appendix B: Trained Model / Weights Link

No model was trained or fine-tuned in this project, so there are no trained weights to publish. Every model used is a pretrained model that is downloaded from its publisher or reached through its provider's API, as listed in Table B.1. The two small models for embedding and re-ranking are downloaded automatically by the sentence-transformers

library the first time the application starts, the local language model is installed with Ollama, and the hosted models are reached with the API keys in the .env file. Because the system's behaviour depends on these exact models, their identifiers are fixed in the configuration and recorded with every evaluation result, and changing a model therefore requires re-running the evaluation of Chapter 6 rather than reusing its numbers, which is how the two evaluation runs reported in Table 6.1 were produced.

Table B.1 Pretrained models used

Role	Model	Source
Embedding	all-MiniLM-L6-v2	https://huggingface.co/sentence-transformers/all-MiniLM-L6-v2
Re-ranker	cross-encoder/ms-marco-MiniLM-L-6-v2	https://huggingface.co/cross-encoder/ms-marco-MiniLM-L-6-v2
Primary LLM and image reading	claude-sonnet-5-5	Anthropic Claude API
Fallback LLMs	gemini-3.8-flash, gemini-3.5-flash-lite	Google Gemini API
Offline LLM	llama3.1:8b	https://ollama.com/library/llama3.1

Appendix C: Key Code

The key code is explained in Section 5.3. Listing C.1 shows the organisation of the repository and Table C.1 the size and responsibility of each module, measured in lines of code at the time this report was built. The application layer in core/ contains no user-interface code, so every rule described in Chapter 4 can be tested directly, and the presentation layer in app.py, ui/ and views/ only calls functions of core/. The evaluation scripts in evaluation/ use the same functions as the application, so the evaluated pipeline is exactly the pipeline that students use, and the end-to-end checks in tests/ drive the running application in a browser. Listing C.2 shows the blind, order-randomised judging used in the evaluation, which is referred to in Sections 4.4 and 6.6 and is the basis of every accuracy figure reported in Chapter 6 of this report.

Listing C.1 Project structure

```
PersonaAI/
├── app.py              # sign-in, first sign-in, page router
├── views/              # home, notes_qa, quiz, progress, study_plan,
admin
├── ui/                 # styles.py (theme, line icons), shell.py
(sidebar)
├── core/               # config, auth, db, documents, ocr, ingest,
vectorstore,
│                       # rag, llm, quiz, memory, planner, ics,
notify, charts
├── scripts/            # send_reminders.py, setup_demo_ai_cohort.py
├── evaluation/         # build_questions, run_eval,
simulate_adaptive, results/
├── tests/              # 159 tests (pytest, AppTest) + Playwright
browser checks
├── .streamlit/         # config.toml: theme
├── data/               # uploads, ChromaDB, SQLite, backups (git-
ignored)
├── requirements.txt, requirements-dev.txt
└── README.md
```

Table C.1 Modules and their responsibilities

Module	Responsibility	Lines
core/db.py	SQLite schema and queries; topic scores rebuilt after every quiz	556
core/planner.py	Timetable plans, shared sessions, regeneration	426
core/quiz.py	Chapters, MCQ generation with a schema, validation, grading	383
core/llm.py	Claude, Gemini and Ollama with timeouts, retries and logging	299
core/documents.py	PDF/DOCX/PPTX/PPT/TXT extraction, cleaning, chunking	258
core/ocr.py	Image-only pages and slides read by Claude vision	182
core/rag.py	Question splitting, two-stage retrieval, grounded answers	181
core/ingest.py	Upload pipeline and topic extraction with coverage	177
core/auth.py	Passwords, one-time codes, lockout	162
core/memory.py	Topic statistics, classification, adaptive allocation	140
core/vectorstore.py	Embeddings and ChromaDB search filtered by user and cohort	118
core/notify.py, core/ics.py	Email updates and calendar files	173
app.py, ui/, views/	Sign-in, router, sidebar, theme and the seven pages	1,776

Listing C.2 Blind, order-randomised judging of the two answers (evaluation/run_eval.py)

```
def judge(q: dict, rag_text: str | None, no_rag_text: str) -> dict:
    """Blind, order-randomised grading of both answers in one call.
A RAG answer of
    "not found" is graded wrong without asking the judge."""
    from core import llm

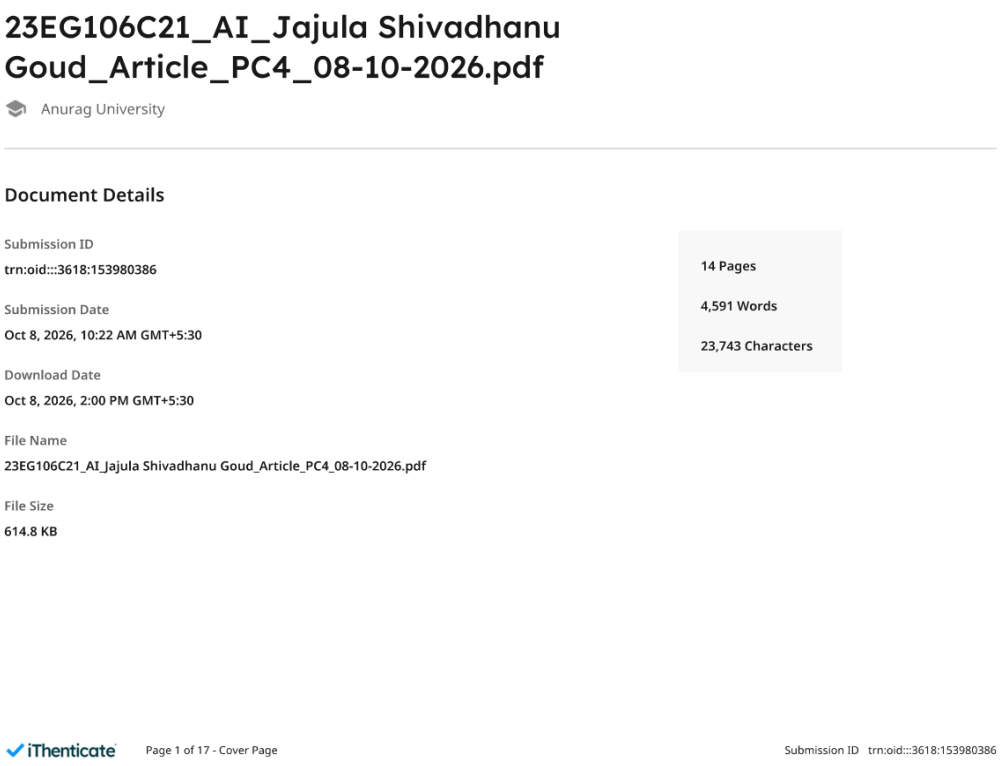
    rag_shown = rag_text if rag_text is not None else "I couldn't
find this in the notes."
```

```
    pair = [("rag", rag_shown), ("no_rag", no_rag_text)]
    random.shuffle(pair)
    prompt = JUDGE_PROMPT.format(
        question=q["question"], reference=q["reference_answer"],
        answer1=pair[0][1], answer2=pair[1][1],
    )
    try:
        reply, judge_model = llm.generate_with_model(prompt,
json_mode=True, temperature=0)
    except llm.LLMError as exc:
        raise QuotaStop(str(exc)[:300]) from exc
    data = json.loads(re.sub(r"^```(?:json)?|```$", "",
reply.strip(), flags=re.M).strip())

    out = {"judge_model": judge_model}
    for slot, (name, _) in zip(("answer1", "answer2"), pair):
        item = data.get(slot) or {}
        verdict = str(item.get("verdict", "")).lower().strip()
        out[f"{name}_verdict"] = verdict if verdict in VERDICTS
else "wrong"
        out[f"{name}_reason"] = str(item.get("reason", "")).strip()
    if rag_text is None:
        out["rag_verdict"], out["rag_reason"] = "wrong",
"Abstained: answer not found in the notes."
    return out
```

Appendix D: Plagiarism Report

The similarity and AI-writing checks were carried out with iThenticate through Anurag University on 8 October 2026, on the submission "23EG106C21_AI_Jajula Shivadhanu Goud_Article_PC4_08-10-2026.pdf" (14 pages, 4,591 words, submission ID trn:oid:::3618:153980386). The similarity report in Figs. D.1 and D.2 shows an overall similarity of 0 per cent, with no matches in internet sources, publications or submitted student papers, no match groups of any kind and no integrity flags, well below the department's limit of 20 per cent. The AI-writing report in Figs. D.3 and D.4 shows "*%", which iThenticate uses when the detected share is below its 20 per cent reporting threshold, so no passages were surfaced. The cover and overview pages of both reports are reproduced below; the complete reports are available from the project team on request.



23EG106C21_AI_Jajula Shivadhanu Goud_Article_PC4_08-10-2026.pdf

Anurag University

Document Details

Submission ID
trn:oid:::3618:153980386

Submission Date
Oct 8, 2026, 10:22 AM GMT+5:30

Download Date
Oct 8, 2026, 2:00 PM GMT+5:30

File Name
23EG106C21_AI_Jajula Shivadhanu Goud_Article_PC4_08-10-2026.pdf

File Size
614.8 KB

14 Pages

4,591 Words

23,743 Characters

Fig. D.1 Similarity report: cover page

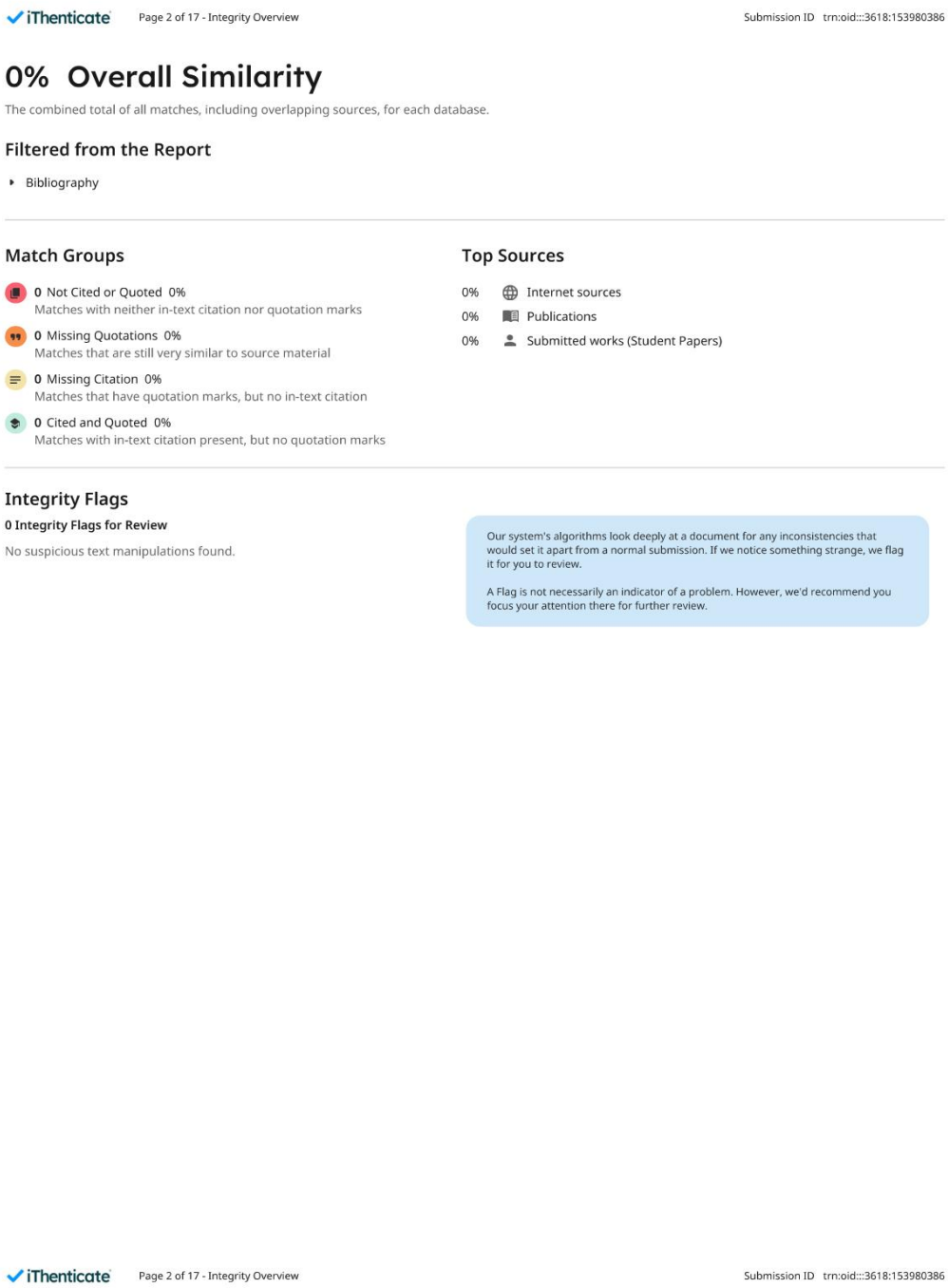
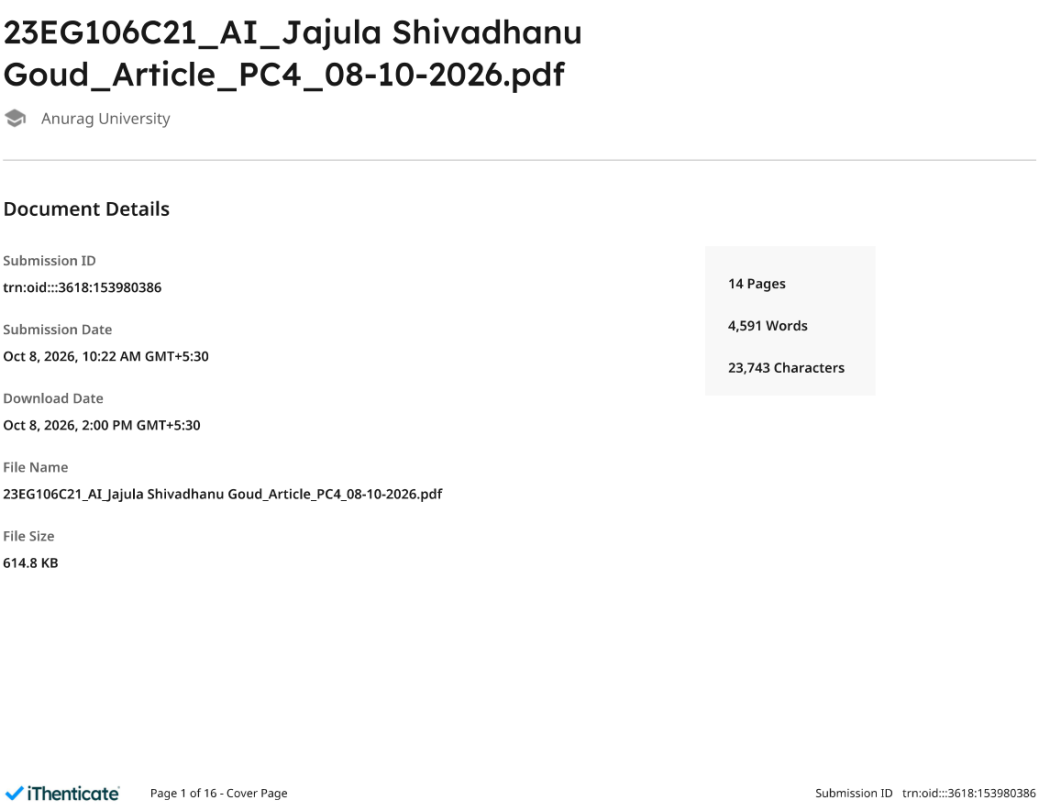


Fig. D.2 Similarity report: integrity overview



23EG106C21_AI_Jajula Shivadhanu Goud_Article_PC4_08-10-2026.pdf

Anurag University

Document Details

Submission ID
trn:oid:::3618:153980386

Submission Date
Oct 8, 2026, 10:22 AM GMT+5:30

Download Date
Oct 8, 2026, 2:00 PM GMT+5:30

File Name
23EG106C21_AI_Jajula Shivadhanu Goud_Article_PC4_08-10-2026.pdf

File Size
614.8 KB

14 Pages

4,591 Words

23,743 Characters

iThenticate Page 1 of 16 - Cover Page Submission ID trn:oid:::3618:153980386

Fig. D.3 AI-writing report: cover page

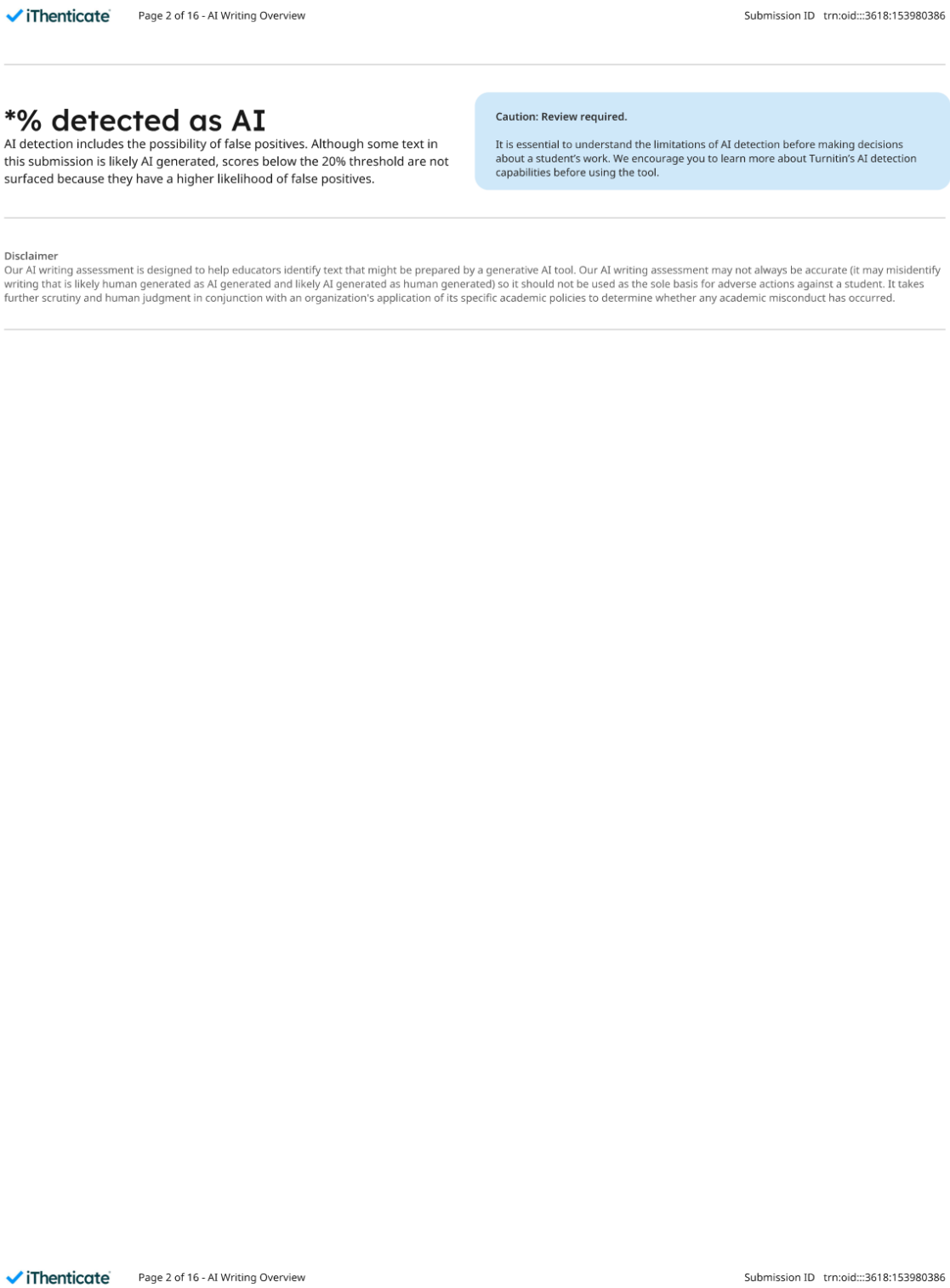


Fig. D.4 AI-writing report: overview

Appendix E: Publications / Certificates

The research paper entitled ‘Evaluating Provenance-Aware Persistent Memory for Cross-Session Personalization in LLM Assistants’ was prepared by Jajula Shivadhanu Goud (23EG106C21), Kothapally Sai Mani Prasad (23EG106C34), and Baddam Supreeth Reddy (23EG106C05) under the guidance of Dr. M. Trupthi, Professor, Department of Artificial Intelligence, Anurag University, Hyderabad, Telangana, India. The manuscript follows a single-column research-article format and includes an abstract, keywords, numbered sections and numbered references. It was submitted for institutional similarity checking on 8 October 2026 under submission ID trn:oid:::3618:153980386 (file “23EG106C21_AI_Jajula Shivadhanu Goud_Article_PC4_08-10-2026.pdf”, 14 pages, 4,591 words). The corresponding iThenticate results, an overall similarity of 0 per cent and an AI-writing score below the 20 per cent reporting threshold, are documented in Appendix D and apply to the research article only, not to the complete mini-project report.

The manuscript has been prepared and subjected to similarity checking. No supporting evidence of submission to a conference or journal, acceptance, or publication was provided; therefore, none of these statuses is claimed. No publication certificate is included.